Bifrost: Empowering Pretrained Language Model with Fallibility Representation for Log-based Fault Diagnosis

ANONYMOUS AUTHOR(S)

Log-based fault diagnosis is crucial for runtime debugging and maintenance. Existing fault diagnosis methods use language models pre-trained on natural language (PLMs) for log representation. However, system faults are reflected in the multi-level structure of system logs. PLMs pre-trained on natural language struggle to comprehensively capture multi-level fault information, failing to meet the requirements of fault diagnosis. We refer to this information as fallibility representations. To address this problem, we propose a novel log representation learning method, Bifrost. It draws inspiration from the log analysis experience of Site Reliability Engineers and meticulously designs strategies based on self-supervised contrastive learning to learn the fallibility representations of logs. Across three public systems and one industrial ML-as-a-Service system, the log representations produced by Bifrost outperform existing PLMs by average margins of 9.83% in F1 for anomaly detection, 18.28% in HR@k for root cause localization, and 20.88% in Macro-F1 for fault identification. Website: https://bifrost4log.github.io/.

CCS Concepts: • **Software and its engineering → Software testing and debugging**.

Additional Key Words and Phrases: Runtime Debugging, Fault Diagnosis, Log Analysis, Language Models

1 Introduction

> ***"You see, but you do not observe."*** — Arthur Conan Doyle, *A Scandal in Bohemia*

Modern software systems are composed of many interacting components and evolve rapidly, which complicates their execution logic and expands the space of potential faults [25, 27, 28, 33, 46]. Even brief outages can incur substantial losses. For example, a one-hour Prime Day disruption at Amazon has been estimated to cost up to 100 million USD [51]. To reduce time-to-recovery and improve availability, providers must quickly identify root causes of production anomalies. Fault diagnosis therefore provides fundamental guidance for debugging and recovery, improving overall system reliability [14, 45, 56].

Logs record the operational status of a system and are a vital source for diagnosing faults in software systems. In recent years, log-based fault diagnosis for software systems has been extensively studied [4, 12, 13, 15, 31, 37, 41, 48, 50, 52, 54, 57, 59]. Existing log-based fault diagnosis methods are typically divided into three main stages. 1) Log parsing [10, 12, 40], which involves converting semi-structured text logs into structured log events. 2) Log representation [8, 21, 29], where the parsed log events are transformed into numerical vectors; recent research predominantly utilizes Pre-trained Language Models (PLMs) for this purpose. 3) Fault diagnosis [5, 7, 22], a stage that involves constructing specific models to diagnose observed system logs, encompassing three main tasks: Anomaly Detection (AD) [19, 50], Root Cause Localization (RCL) [1, 44], and Fault Identification (FI) [5, 39]. In this paper, we primarily focus on the second stage: log representation.

Although existing PLM-based log representation methods have achieved some success, PLMs failing to meet the requirements of fault diagnosis tasks [8, 21, 29]. LLMeLog [8] conducted an empirical study which proved that log representations encoded by PLMs suffer from issues such as content missing, semantic deflection, and tendency lacks, leading to a 19.97% decrease in anomaly detection. KnowLog [29] pointed out that PLMs have significant deficiencies in understanding the abbreviations, context, and style of logs. PreLog [21] analyzed the inadequacy of PLMs in understanding log evolution. Overall, leveraging the full potential of PLMs for software system fault diagnosis still faces significant challenges.

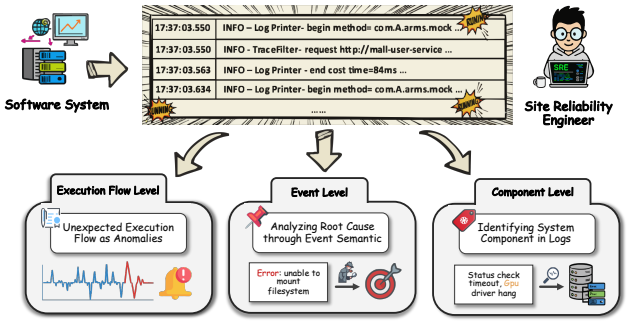


Fig. 1. Practical log analysis experience of SREs.

In our view, the root cause of these issues is the inherent structural differences between logs and natural language. The characteristics of system faults do not exist in a singular semantic form but are manifested through the multi-level structure of logs. Therefore, relying solely on models pre-trained on natural language makes it difficult to comprehensively capture and understand this complex structural information, thus impeding effective fault diagnosis.

In fact, in real-world software system reliability maintenance practices, Site Reliability Engineers (SREs) typically analyze logs from three levels: execution flow, event, and component, to gain insights into system faults, as shown in Figure 1. 1) Execution Flow Level: SREs focus on scrutinizing execution flows that deviate from historical patterns. Such unexpected execution paths are considered strong indicators of nomalies [4, 31]. 2) Event Level. In production practice, SREs abstract a single log entry into an independent system event. The specific semantics carried by each event serve as the key basis for them to determine the cause of a fault [8]. 3) Component Level: Log records contain fine-grained information about system components (e.g., GPU, disk). By deeply analyzing this information, SREs can localize the fault to specific hardware or software units [5].

We term the information that SREs obtain from the multi-level analysis of logs as fallibility representations. PLMs, operating in the form of masked language modeling, can only encode the natural language information of system logs while neglecting the fault-related fallibility representations, leading to poor performance of the encoded log representations in various fault diagnosis tasks. To investigate the capability of PLMs in encoding fallibility representations, this paper conducts an empirical study on existing PLMs using three log datasets. The results indicate that existing PLMs are indeed incapable of encoding these fallibility representations, which leads to unsatisfactory fault diagnosis performance.

Solving this problem requires a comprehensive learning of the fallibility representations of system logs. However, this is not straightforward. Firstly, fallibility cannot be directly obtained from system logs. How can learning objectives be designed to effectively learn different types of fallibility representations? Secondly, the learning processes for different types of fallibility representations may interfere with each other. How can a learning strategy be designed to comprehensively learn fallibility representations?

To address these issues and unlock the potential of PLMs in fault diagnosis tasks, in this paper, we propose Bifrost, an innovative log representation learning method designed to enhance the capabilities of PLMs in fault diagnosis by learning fallibility representations from the continuously generated system logs of software systems. To tackle the first problem, Bifrost first meticulously designs three training tasks: Execution Flow Prediction (EFP), Abnormal Event Discrimination (AED), and System Component Perception (SCP). These tasks, based on the paradigm of self-supervised contrastive learning, are capable of learning different types of fallibility representations from system logs. To address the second problem, we first conducts an error analysis of the learning

process for fallibility representations. We discovered that due to the inherent random sampling process in the fallibility learning process, an error is generated at each step of the optimization trajectory, which ultimately accumulates in the model's weight parameters, a phenomenon we term the fallibility jitter problem. To solve the fallibility jitter problem, BIFROST proposes the Co-Anchored Fallibility Representation Learning (CARL) strategy. The key idea of this strategy is to use system logs from real execution flows as representation space anchors for multi-objective representation learning, thereby reducing weight errors during the learning process.

To evaluate the effectiveness of BIFROST, we conducted exhaustive evaluations on three of the most widely used public datasets [34, 60] for log-based fault diagnosis—BGL, Hadoop, and Thunderbird—as well as on dataset collected from an industrial ML-as-a-Service (MLaaS) system, Platform-X, covering the three fault diagnosis tasks of AD, RCL, and FI. The experimental results demonstrate that the log representations generated by BIFROST achieve advanced performance in fault diagnosis tasks. Compared to PLMs with a similar number of parameters, BIFROST achieved an average advantage of 20.81% across the three tasks. Compared to Large Language Models (LLMs) with tens of times more parameters than BIFROST, BIFROST also achieved an average advantage of 5.80%. This evidence underscores the potent role of BIFROST and fallibility representations in fault diagnosis tasks.

In summary, our contributions are as follows:

- We analyzed the log analysis experience of SREs and identified three types of information closely related to failure, which we term fallibility representations.
- We conducted an empirical study, the results of which show that existing PLM-based log representation methods perform poorly in encoding fallibility representations, leading to suboptimal failure diagnosis.
- We propose BIFROST, a representation learning method that enhances the failure diagnosis capabilities of PLMs by learning fallibility representations.
- Quantitative, qualitative and theoretical evaluations on four datasets demonstrate the effectiveness of BIFROST and fallibility representations in failure diagnosis.

2 Task Formulation

Our objective is to conduct fault diagnosis for software systems based on system logs. Consider a large-scale software system, within an observation window of length w, we have a log sequence defined as $\mathbf{x}=[l_1,l_2,\ldots,l_w]$, where l_i is a log event observed at a certain time step. The fault diagnosis aims to establish a framework that processes the input log sequence and outputs 1) the operational status of the system, and upon detection of an anomaly, further outputs 2) the root cause logs and 3) the fault type. This task consists of three sub-tasks:

- **Anomaly Detection (AD)** [8, 9, 11]: Given an observed log sequence $\mathbf{x}$, a detector $\mathcal{D}$ is used to predict the presence of an anomaly. Its output, y, is a binary variable indicating whether the system is normal (0) or anomalous (1), i.e., $y=\mathcal{D}(\mathbf{x})\in\{0,1\}$.
- **Root Cause Localization (RCL)** [38, 43, 55]: If the system state is determined to be anomalous, a localizer $\mathcal{R}$ is activated to evaluate the probability of each log l_i in the input sequence $\mathbf{x}$ being the root cause. That is, $\mathcal{R}(\mathbf{x})=[p_1,p_2,\ldots,p_w]$, where $p_i\in[0,1]$ represents the probability that log l_i is the root cause.
- **Fault Identification (FI)** [5, 39]: After confirming a system anomaly, a multi-class classifier $\mathcal{M}$ is employed to identify the specific type of the fault. It is assumed that the potential fault types belong to a predefined, finite set $\mathcal{U}=\{u_1,u_2,\ldots,u_{|\mathcal{U}|}\}$. The objective of the $\mathcal{M}$ is to predict the fault type corresponding to the current log sequence, i.e., $\mathcal{M}(\mathbf{x})\in\mathcal{U}$.

3 Empirical Study

In section, we conduct an empirical study on existing PLM-based log representation methods on three systems [34, 60]: Thunderbird, BGL, and an industrial MLaaS system Platform-X. We manually examined the capability of the four most widely used PLMs [3, 18, 23, 36] to encode fallibility representations. **Section 5.1** provides a detailed introduction to the baseline models and the systems.

3.1 How effective are existing PLMs in identifying unexpected execution flows?

In this section, we evaluate the capability of existing PLMs to identify unexpected execution flows in the form of predicting the next event. We first collect a normal execution flow of the system, denoted as $L = [l_1, l_2, ..., l_T]$. Then, we use a time window w to partition L into execution flows of length w, represented as $\mathbf{x}_t = [l_{t-w}, ..., l_{t-1}]$. We use PLMs, denoted as $f_\theta(\cdot)$, for encoding to obtain a sequence of log representations $f_\theta(\mathbf{x}_t)$. This sequence is subsequently fed into a sequential neural network $g_\phi(\cdot)$ to predict the next log in the execution flow: $\hat{e}_t = g_\phi(f_\theta(\mathbf{x}_t))$. Our training objective is to maximize the cosine similarity $s(\hat{e}_t, e_t)$ between $\hat{e}_t$ and the embedding of the next true log in the execution flow, $e_t = f_\theta(l_t)$. Here, the parameters θ of the PLM f_θ are frozen to evaluate the representation quality of the original PLMs. After training is complete, we use unexpected execution flows for evaluation. For an unexpected event representation $\tilde{e}_t$, a smaller value of $s(\hat{e}_t, \tilde{e}_t)$ indicates a stronger capability of the PLM to identify unexpected execution flows.

We use the absolute cosine similarity, *Avg_Sim*, and the relative rank similarity, *Avg_Rank*, to quantify the similarity:

- $Avg_Sim = \mathbb{E}[s(\tilde{e}_i, \hat{e}_i)]$
- $Avg_Rank = \mathbb{E}\left[1 + \sum_{e_j \in \mathbf{E}(L)} \mathbb{I}\left(s(\hat{e}_i, \tilde{e}_i) \leq s(\hat{e}_i, e_j)\right)\right]$

where $\mathbb{E}$ denotes the expected value and $\mathbb{I}$ is the indicator function. A lower *Avg_Sim* suggests a stronger ability of the PLM to identify unexpected execution flows, whereas for *Avg_Rank*, the opposite is true. The experimental results on the Thunderbird system logs are shown in Table 1. It is readily apparent that existing methods consider unexpected execution flows to have a high similarity to historical execution flows, which indicates that the capability of existing PLMs to identify unexpected execution flows is not ideal.

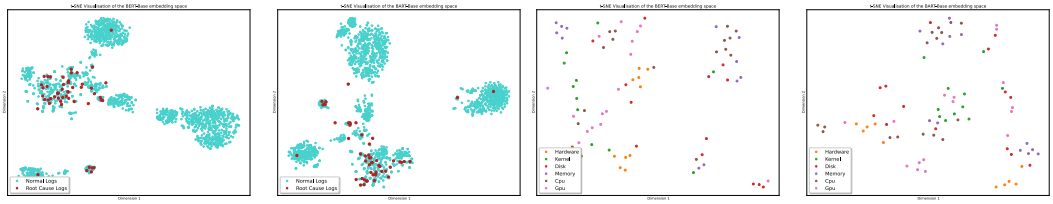
Table 1. Representational similarity between expected and unexpected events in PLMs.

Embedding Model	**Thunderbird**	
Metrics	**Avg_Sim ↓**	**Avg_Rank ↑**
Glove-300d [36]	78.08	13.49
BERT-Base [3]	89.62	21.69
ALBERT-Base [18]	92.22	21.05
BART-Base [23]	81.03	17.16
Average	88.10	20.94

3.2 Can existing PLMs discriminate anomalous system events?

This section investigates the discriminative ability of existing PLMs for anomalous system events. Limited by space constraints, we select the most representative BERT-Base [3] and BART-Base [23] as baseline models, and choose the BGL system for our study. We use PLMs to encode the logs from the BGL system and then visualize these representations using t-SNE in Figure 2a and Figure 2b.

It is easy to observe that in the log representations obtained by existing PLMs, normal system events and anomalous system events are heavily stacked together, and the distribution of anomalous system events is relatively dispersed. This evidence indicates that existing PLMs do not have the ability to distinguish anomalous semantics within logs, resulting in non-ideal log representations.



(a) BERT-Base (b) BART-Base (c) BERT-Base (d) BART-Base

Fig. 2. Fig. 2a and Fig. 2b show the system-event representations generated by BERT-Base and BART-Base on BGL, respectively. Fig. 2c and Fig. 2d show the corresponding system-component representations on Platform-X.

3.3 Can existing PLMs perceive different system components?

This section investigates the capability of existing PLMs to perceive different system components. Due to space constraints, we select the most representative BERT-Base and BART-Base as baseline models and choose Platform-X as the system for our study. Similar to Section 3.2, we use PLMs to encode the logs from the Platform-X system and then visualize these representations using t-SNE, with the results shown in Figure 2c and 2d. It is easy to observe that in the log representations obtained by existing PLMs, the representations of different system components are stacked together. This indicates that it is difficult for existing PLMs to perceive different system components from the raw logs.

Summary. PLM-based log representation methods are only pre-trained on natural language and overlook the multi-level structure of logs, leading to: 1) an inability to identify unexpected execution flows, 2) insensitivity to anomalous system events, and 3) a failure to separate different system components. Thus resulting in suboptimal fault diagnosis performance.

4 Methodology

4.1 Overview

This paper introduces BIFROST, an innovative log representation learning method designed to enhance the capabilities of PLMs in fault diagnosis. While log-based fault diagnosis is crucial for maintaining the reliability of software systems, existing PLM-based methods primarily encode semantic information from logs. They often fail to perceive system failure patterns embedded within the multi-level structure of logs, leading to suboptimal diagnostic performance. To bridge this gap, Bifrost draws inspiration from the analytical strategies of SREs. BIFROST formulate three distinct learning objectives, each targeting a specific type of fallibility representation corresponding to a different level of the log structure. However, influenced by in-batch stochastic sampling, the optimization process of fallibility learning exhibits significant instability. We term this **Fallibility Jitter** and have analyzed the weight error upper bound. To address the fallibility jitter problem, BIFROST proposes the **Co-Anchored Fallibility Representation Learning** strategy. By robustly learning these representations, BIFROST produces log embeddings that are highly effective for fault diagnosis tasks. The overall pipeline of BIFROST is illustrated in Figure 3.

4.2 Fallibility Representation for Log-based Fault Diagnosis

BIFROST aims to learn **fallibility representations** from system logs, thereby producing embeddings tailored for fault diagnosis. Since such representations are latent and cannot be directly supervised, we operationalize fallibility learning with three self-supervised contrastive objectives: Execution Flow Prediction (EFP), Abnormal Event Discrimination (AED), and System Component Perception (SCP).

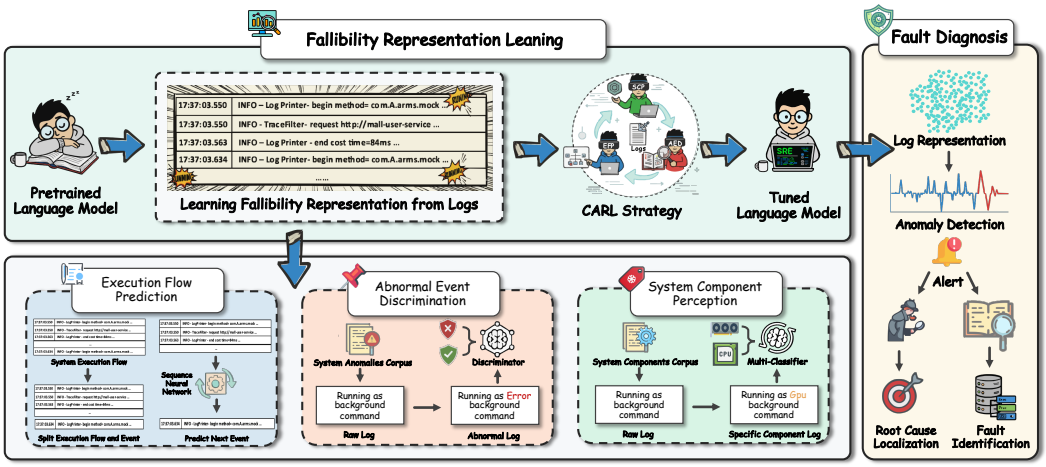


Fig. 3. The pipeline of Bifrost. Bifrost is initialized from the PLM BART-Base. It is then trained via a carefully designed fallibility representation learning process, which learns log fallibility representations from logs collected during normal system operation. After training, Bifrost encodes logs for fault diagnosis.

4.2.1 ***Self-Supervised Learning of Fallibility Representations.*** Bifrost learns fallibility representations for log-based fault diagnosis. Since fallibility is a latent property spanning multiple log-structure levels, direct scalable supervision is difficult to obtain. We view fallibility as *relational* in the embedding space: instances preserving a target level-specific consistency should be close, whereas those violating it should be separated. For example, logs following a normal execution flow should cluster, while logs linked to unexpected behaviors, anomalous event semantics, or different system components should be farther apart. This motivates a self-supervised contrastive objective [2, 17] that provides proxy supervision for learning fallibility representations.

Let a chronological log stream from normal operation be $\mathbf{L} = [l_1, \ldots, l_T]$. An embedding model $f_\theta(\cdot)$ maps a log message or a log-message sequence to $\mathbb{R}^d$. We use cosine similarity $s(\cdot,\cdot)$ and a temperature hyperparameter τ. We use the term *instance* to denote a training input (a single log message or a context window). Given an anchor instance v, we sample a positive instance $v^+ \in \mathcal{P}(v)$ that preserves the target level-specific consistency, and sample K negative instances $\{\tilde{v}_k\}_{k=1}^{K} \subset \mathcal{N}(v)$ that violate this consistence. We then define the InfoNCE loss [35]:

$$\mathcal{L}_{\mathrm{CL}}\big(v, v^+, \mathcal{N}(v)\big) = -\log \frac{\exp(s(f_\theta(v), f_\theta(v^+))/\tau)}{\exp(s(f_\theta(v), f_\theta(v^+))/\tau) + \sum_{\tilde{v} \in \mathcal{N}(v)} \exp(s(f_\theta(v), f_\theta(\tilde{v}))/\tau)}. \tag{1}$$

We minimize the mini-batch average of $\mathcal{L}_{\mathrm{CL}}$ to optimize θ. Here, $\mathcal{P}(\cdot)$ and $\mathcal{N}(\cdot)$ specify instance construction rules that preserve or violate the target level-specific consistency. Using this shared contrastive-learning template, we instantiate three objectives aligned with the multi-level log structure: EFP, AED, and SCP. Each objective defines the anchor instance v and the corresponding $\mathcal{P}(\cdot)$ and $\mathcal{N}(\cdot)$ to target a specific level of fallibility.

4.2.2 ***Execution Flow Prediction (EFP).*** In real-world practice, SREs pay close attention to execution flows that deviate from established historical patterns. Such unexpected execution paths are considered strong indicators of system anomalies. Consequently, the ideal log representation must be capable of identifying these deviations. To this end, we propose the EFP task, which learns the patterns of normal system execution flows in a predictive manner.

To assess whether an upcoming event is compatible with the normal execution flow, EFP uses the preceding execution context as the anchor. At time t, we form a context-window instance $\mathbf{x}_t =$

$[l_{t-w},\ldots,l_{t-1}]$ and obtain its embedding via $f_\theta(\cdot)$. An EFP-specific prediction head $g_\phi(\cdot)$, implemented as a lightweight sequence model, projects the context embedding into the log embedding space:

$$\hat{e}_t = g_\phi(f_\theta(\mathbf{x}_t)). \tag{2}$$

The predictive anchor $\hat{e}_t$ summarizes the historical flow and is used to assess whether the subsequent event follows the normal execution flow.

To avoid template memorization, we sample a random in-window message $l_i \in \mathbf{x}_t$ and construct positives as its semantics-preserving perturbations, reflecting the flow state rather than the exact next message l_t. Let $\mathcal{P}_{\text{EFP}}(l_i)$ be the set of variants produced by lightweight perturbations (random word deletion, synonym replacement, and adjacent word swapping), and we sample $l_i^+ \in \mathcal{P}_{\text{EFP}}(l_i)$ as the positive instance. This preserves event semantics, mitigates template overfitting, and improves generalization to unseen normal execution flows [16, 26].

To expose execution-flow violations without explicit labels, we contrast the predictive anchor against negative continuations that break execution-flow consistency. Given a context $\mathbf{x}_t$ from normal operation, the observed next event l_t serves as a flow-consistent continuation. We construct negatives by injecting failure semantics from a fault lexicon $\mathcal{A}$ into l_t, yielding counterfactual continuations that introduce failure cues and are unlikely under the normal execution flow. Specifically, we sample K negatives $\{\tilde{l}_{t,k}\}_{k=1}^K \subset \mathcal{N}_{\text{EFP}}(l_t)$ by inserting fault phrases $a_k \in \mathcal{A}$ into l_t:

$$\tilde{l}_{t,k} = l_t \oplus a_k, \quad a_k \in \mathcal{A}, \tag{3}$$

where $\oplus$ inserts a_k at a random position and $\mathcal{A}$ is described in Sec. 4.2.5. EFP then optimizes:

$$\min_\theta \mathcal{L}_{\text{EFP}}\left(\hat{e}_t, l_i^+, \{\tilde{l}_{t,k}\}_{k=1}^K\right) = -\frac{1}{T-w}\sum_{t=w+1}^{T} \log \frac{\exp(s(\hat{e}_t, f_\theta(l_i^+))/\tau)}{\exp(s(\hat{e}_t, f_\theta(l_i^+))/\tau) + \sum_{k=1}^K \exp(s(\hat{e}_t, f_\theta(\tilde{l}_{t,k}))/\tau)}. \tag{4}$$

This objective aligns $\hat{e}_t$ with flow-consistent evidence and separates it from flow-violating counterfactuals, thereby encoding execution-flow level-specific consistency in the learned representations.

4.2.3 ***Abnormal Event Discrimination (AED).*** At the event level, SREs abstract individual log entries into discrete system events and reason about their semantics to identify the nature and cause of faults. Thus, beyond capturing flow-level deviations, an effective log representation should also separate normal and abnormal semantics at the event level. To this end, BIFROST includes the AED objective, which directly contrasts normal event semantics against anomaly-like variants.

In AED, each log message from normal operation $l_t \in \mathbf{L}$ is treated as an event-level anchor instance. For a given anchor l_t, we construct a positive instance $l_t^+ \in \mathcal{P}_{\text{AED}}(l_t)$ by applying the same semantics-preserving perturbations as in EFP. These perturbations preserve the event intent while encouraging robustness to superficial lexical changes, so that variants of the same normal event cluster in the embedding space.

To model abnormal semantics, we construct negative instances by injecting failure-related phrases from the fault lexicon $\mathcal{A}$ into the anchor event. Specifically, we sample K negatives $\{\tilde{l}_{t,k}\}_{k=1}^K \subset \mathcal{N}_{\text{AED}}(l_t)$ of the form

$$\tilde{l}_{t,k} = l_t \oplus a_k, \quad a_k \in \mathcal{A}, \tag{5}$$

where $\oplus$ inserts a_k at a random position and $\mathcal{A}$ is described in Sec. 4.2.5. This injection yields counterfactual variants that introduce failure cues and are unlikely under normal operation. Given the anchor l_t, the positive l_t^+, and the negatives $\{\tilde{l}_{t,k}\}_{k=1}^K$, AED instantiates the contrastive loss as

$$\min_\theta \mathcal{L}_{\text{AED}}\left(l_t, l_t^+, \{\tilde{l}_{t,k}\}_{k=1}^K\right) = -\frac{1}{T-w}\sum_{t=w+1}^{T} \log \frac{\exp(s(f_\theta(l_t), f_\theta(l_t^+))/\tau)}{\exp(s(f_\theta(l_t), f_\theta(l_t^+))/\tau) + \sum_{k=1}^K \exp(s(f_\theta(l_t), f_\theta(\tilde{l}_{t,k}))/\tau)}. \tag{6}$$

This objective pulls embeddings of semantics-preserving variants of a normal event together and pushes them away from failure-injected variants, thereby capturing event-level fallibility.

4.2.4 ***System Component Perception (SCP)***. Log entries often contain explicit references to system components (e.g., GPU, disk, memory). By analyzing such component cues, SREs can localize faults to specific software or hardware units and narrow the diagnostic scope. Therefore, effective log representations should also be sensitive to component identity. The SCP objective is designed to encode this component-level fallibility in the embedding space.

We assume a predefined component vocabulary $\mathcal{C}=\{c_1,\ldots,c_{|\mathcal{C}|}\}$ that covers common component types in distributed systems (e.g., `cpu`, `memory`), the full list is given in Appendix A. This reflects the practical observation that many failures manifest at the component granularity, and these keywords provide a simple and portable inductive bias. For each log message l_t from normal operation, we randomly sample a component keyword $c_j \in \mathcal{C}$ and construct an anchor instance by inserting c_j into l_t:

$$l_{t,j} = l_t \oplus c_j, \tag{7}$$

where $\oplus$ denotes string insertion at a random position. Thus, $l_{t,j}$ can be viewed as a log instance explicitly associated with component c_j.

To encourage component-consistent clustering and robustness to surface variations, we construct positives as semantics-preserving variants of the anchor. Specifically, we define $\mathcal{P}_{\text{SCP}}(l_{t,j})$ as augmentations obtained by applying the lightweight perturbations from EFP and AED to $l_{t,j}$, and sample a positive instance $l^+_{t,j} \in \mathcal{P}_{\text{SCP}}(l_{t,j})$. These augmentations keep the component keyword c_j while making representations invariant to minor lexical changes.

To separate different components, we construct negative instances by injecting alternative component keywords into the same base log. For each anchor $l_{t,j}$, we randomly sample Z negative instances $\{\tilde{l}_{t,j,z}\}_{z=1}^{Z} \subset \mathcal{N}_{\text{SCP}}(l_t, c_j)$ of the form

$$\tilde{l}_{t,j,z} = l_t \oplus c_i, \quad c_i \in \mathcal{C} \setminus \{c_j\}, \tag{8}$$

again using $\oplus$ to denote string insertion. These negatives are logs sharing the same base content but associated with different components and should be distinguished at the component level.

Given the anchor $l_{t,j}$, the positive $l^+_{t,j}$, and the negatives $\{\tilde{l}_{t,j,z}\}_{z=1}^{Z}$, SCP then optimizes:

$$\min_\theta \mathcal{L}_{\text{SCP}}\left(l_{t,j}, l^+_{t,j}, \{\tilde{l}_{t,j,z}\}_{z=1}^{Z}\right) = -\frac{1}{T-w}\sum_{t=w+1}^{T} \log \frac{\exp(s(f_\theta(l_{t,j}), f_\theta(l^+_{t,j}))/\tau)}{\exp(s(f_\theta(l_{t,j}), f_\theta(l^+_{t,j}))/\tau) + \sum_{k=1}^{K} \exp(s(f_\theta(l_{t,j}), f_\theta(\tilde{l}_{t,j,z}))/\tau)}, \tag{9}$$

This objective pulls together instances associated with the same component keyword and pushes them away from instances associated with different components, thereby capturing component-level fallibility in the learned representations.

4.2.5 ***Fault Lexicon Construction***. The negative generators in EFP and AED inject failure semantics by inserting phrases from a fault lexicon $\mathcal{A}$. A manually crafted lexicon is labor-intensive to maintain and may not reflect how failures are described in the target logs. We therefore construct $\mathcal{A}$ from the training corpus via two stage: *candidate recall* followed by *reranking*.

Stage 1: Candidate Recall. The goal of this stage is to retrieve a broad set of corpus-specific phrases that are semantically compatible with failure-indicating positions in system logs. Following the standard log-level taxonomy in SRE practice, we treat tokens such as `warning`, `error`, and `fatal` as seed anchors of failure-related semantics. We tokenize the training logs and the WordNet [32] lexicon and merge them into a unified retrieval vocabulary. For each anchor, we embed all its word instances with Qwen3-Embedding-0.6B [49] and use their mean vector as the query representation. We then embed every entry in the retrieval vocabulary and rank them by cosine

similarity to the query, retrieving the top-300 as the candidate pool Ω. In our implementation, we use Qwen3-Embedding-0.6B [49] for embedding-based retrieval.

Stage 2: Reranking. High-frequency phrases are not necessarily fault-related; many correspond to benign states or generic descriptions. To focus on failure-oriented semantics, we employ a reranker $r_\psi(q,\kappa) \in [0,1]$ that scores the semantic relatedness between an anchor query q and a candidate phrase $\kappa \in \Omega$. We set q to the concatenation of the three log-level anchors, `warning`, `error`, `fatal`, score each candidate in Ω, and select the top-m phrases as the final fault lexicon:

$$\mathcal{A} = \mathrm{Top}_m\big(\{\kappa \in \Omega\}, r_\psi(q,\kappa)\big). \tag{10}$$

We instantiate $r_\psi(\cdot,\cdot)$ with Qwen3-Reranker-0.6B [49] and set m=30 by default. This pipeline yields a compact, corpus-grounded lexicon of failure-related phrases, supporting realistic and diverse negative instance construction for fallibility learning.

4.3 Learning from Fallibility Representation

The goal of BIFROST is to provide an end-to-end framework for fallibility representation learning. We first analyze a vanilla training strategy that jointly optimizes the three objectives. We find that inherent stochastic sampling induces significant optimization instability, a phenomenon we term **fallibility jitter**, which prevents the model from converging to an ideal representation space. To mitigate this issue, we propose the **CARL** strategy, which enables more robust and efficient representation learning by anchoring optimization on real system events.

4.3.1 ***Fallibility Jitter in Representation Learning.*** To learn fallibility representations in an end-to-end manner, the vanilla strategy jointly optimizes the three objectives. At optimization step s, we sample a time index t and denote the log at time t by l_t, together with its context window $\mathbf{x}_t = [l_{t-w},\ldots,l_{t-1}]$, a log message $l_i \in \mathbf{x}_t$ for EFP, and a component keyword $c_j \in C$, and construct the SCP anchor as $l_{t,j} = l_t \oplus c_j$. The vanilla loss is:

$$\mathcal{L}_{\text{Vanilla}} = \lambda_1 \mathcal{L}_{\text{EFP}}(\hat{e}_t, l_i^+, \{\tilde{l}_{t,k}\}_{k=1}^K) + \lambda_2 \mathcal{L}_{\text{AED}}(l_t, l_t^+, \{\tilde{l}_{t,k}\}_{k=1}^K) + \lambda_3 \mathcal{L}_{\text{SCP}}(l_{t,j}, l_{t,j}^+, \{\tilde{l}_{t,j,z}\}_{z=1}^Z), \tag{11}$$

We provide a detailed theoretical analysis of the optimization process for $\mathcal{L}_{\text{Vanilla}}$ in **Appendix B**. Our analysis shows that inherent stochastic sampling causes the parameter trajectory $\mathbf{W}'_s$ driven by stochastic gradients to deviate from the ideal trajectory $\mathbf{W}_s$ driven by the true gradient, leading to suboptimal fallibility representations. We refer to this phenomenon as **fallibility jitter**. In **Appendix C**, we further derive an upper bound on the resulting parameter error:

$$\mathbb{E}[\|\mathbf{W}_S - \mathbf{W}'_S\|] \le \eta \sum_{k=0}^{S-1} (1+\eta L_{\text{Lip}})^{S-1-s} \mathbb{E}_{\mathbf{W}'_s}[\sigma_{\text{Vanilla}}(\mathbf{W}'_s)], \tag{12}$$

where σ_{Vanilla} is the single-step gradient standard deviation, η is the learning rate, L_{Lip} is the Lipschitz constant of the gradient function, and S is the total number of optimization steps. Equation (12) shows that the gradient variance $\sigma^2_{\text{Vanilla}}$ at each step is amplified and accumulated into the final parameter deviation, causing the parameters to oscillate along the convergence path and degrading the quality of fallibility representation learning. This deviation is fundamentally induced by the stochastic sampling inherent in the vanilla optimization process.

4.3.2 ***Co-Anchored Fallibility Representation Learning.*** To mitigate fallibility jitter, we propose the CARL strategy. The key idea is to use the real system event l_t as a common optimization anchor, providing a deterministic center for each stochastic batch and stabilizing the training process. At each optimization step s, we sample a time index t and denote the log at time t by l_t, together with its context window $\mathbf{x}_t = [l_{t-w},\ldots,l_{t-1}]$, a context log $l_i \in \mathbf{x}_t$ for EFP, and a component keyword

$c_j \in C$, and construct the SCP anchor as $l_{t,j} = l_t \oplus c_j$. CARL then redesigns the joint objective as:

$$\begin{aligned}\mathcal{L}_{\text{CARL}} = \lambda_1 \mathcal{L}_{\text{EFP}}(\hat{e}_t, l_t^+, \{\tilde{n}_{t,j,k}\}_{k=1}^{K+Z}) + \lambda_2 \mathcal{L}_{\text{CL}}(l_t^+, l_i^+, \{\tilde{n}_{t,j,k}\}_{k=1}^{K+Z}) \\ + \lambda_3 \mathcal{L}_{\text{AED}}(l_t, l_t^+, \{\tilde{n}_{t,j,k}\}_{k=1}^{K+Z}) + \lambda_4 \mathcal{L}_{\text{SCP}}(l_{t,j}, l_{t,j}^+, \{\tilde{l}_{t,j,z}\}_{z=1}^{Z}),\end{aligned} \tag{13}$$

$$\{\tilde{n}_{t,j,k}\}_{k=1}^{K+Z} \subset \{l_t \oplus a \mid a \in \mathcal{A}\} \cup \{l_t \oplus c \mid c \in C \setminus \{c_j\}\}, \quad \{\tilde{l}_{t,j,z}\}_{z=1}^{Z} \subset \{l_t \oplus c \mid c \in C \setminus \{c_j\}\}. \tag{14}$$

The key design choices are: (1) a revised EFP term that uses the central event l_t^+ as the flow-consistent continuation (the λ_1 term); (2) an auxiliary contrastive constraint $\mathcal{L}_{\text{CL}}$ between l_t^+ and the sampled context log l_i^+ to explicitly handle the stochasticity from context sampling (the λ_2 term); and (3) a shared negative set $\{\tilde{n}_{t,j,k}\}_{k=1}^{K+Z}$ across EFP, $\mathcal{L}_{\text{CL}}$, and AED, which couples their gradients through common negative evidence. We analyze the optimization behavior of $\mathcal{L}_{\text{CARL}}$ in **Appendix D**. The analysis shows that CARL reduces parameter error through two structural mechanisms, thereby achieving more stable and efficient fallibility representation learning:

(1) **Variance isolation:** The variance introduced by sampling l_i is isolated into the independent $\mathcal{L}_{\text{CL}}$ term, which typically has a smaller weight, thus reducing its contribution to the overall gradient variance.
(2) **Negative covariance constraint:** The shared negative set induces a critical negative covariance term, which partially cancels the variance generated by component sampling c_j.

5 Experimental Evaluation

In this section, we evaluate our proposed method by answering the following research questions:

- **RQ1:** How effective is fallibility representation in the log-based fault diagnosis?
- **RQ2:** What are the contributions of different training strategies to Bifrost?
- **RQ3:** How sensitive is Bifrost to the learning of fallibility representations?
- **RQ4:** How capable is Bifrost of encoding the fallibility representation?

5.1 Experimental Setup

5.1.1 ***Datasets.*** We evaluate Bifrost on four log datasets: three public datasets (BGL, Thunderbird, and Hadoop) and one industrial dataset from an MLaaS platform (Platform-X) [34, 60]. BGL and Thunderbird are collected from the Blue Gene/L supercomputer (128K processors) and the Thunderbird supercomputer (9,024 processors), respectively [34]. The Hadoop dataset was collected from a five-node HDFS cluster with 46 CPU cores [60]. Platform-X is an integrated MLaaS platform deployed by a cloud service provider, operating over 6,000 heterogeneous GPUs to execute ML jobs. We collected Platform-X system logs under a 7-day workload. SREs manually classified failures into six component types: hardware, kernel, disk, memory, CPU, and GPU. Due to the scale of Thunderbird, we follow prior work [19, 20] and use the earliest 10 million log messages.

5.1.2 ***Baselines.*** To evaluate Bifrost, we compare against (i) general-purpose text representation baselines in the ∼100M-parameter regime (BERT-Base [3], BART-Base [23], ALBERT-Base [18], and GloVe-300d [36]), (ii) larger embedding-oriented LLMs with >1B parameters (GTE-Qwen-1.5B-Instruct [24], Stella_en_1.5B_v5 [53], E5-mistral-7b-instruct [42], and SFR-Embedding-Mistral [30]), and (iii) a log-pretrained model, PreLog [21].

5.1.3 ***Evaluation Task.*** We evaluate Bifrost for log-based fault diagnosis via three sub-tasks: anomaly detection (AD), root cause localization (RCL), and fault identification (FI). For AD task, We evaluate on BGL, Hadoop, Thunderbird, and Platform-X using Precision, Recall, and F1 [4, 20]. For RCL task, We evaluate on BGL, Thunderbird, and Platform-X (Hadoop lacks root-cause labels/logs for this task) using HR@k, NDCG@k, MAP@k, and MRR with $k \in \{3,5\}$ [7, 44]. For FI

Table 2. Comparison results with baseline methods for root cause localization.

Dataset	Embedding Model	HR@1	HR@3	HR@5	PR@3	PR@5	MAP@3	MAP@5	MRR
BGL	*Pretrained Small Language Models*								
	Glove-300d	40.00 ± 6.00	52.55 ± 5.39	56.29 ± 2.86	44.32 ± 5.02	45.81 ± 3.61	42.41 ± 5.46	43.65 ± 4.77	46.62 ± 5.13
	BERT-Base	58.64 ± 10.0	73.21 ± 12.3	79.83 ± 11.8	61.63 ± 5.87	65.62 ± 6.31	59.65 ± 6.95	61.70 ± 6.43	67.40 ± 8.81
	ALBERT-Base	53.74 ± 0.58	61.24 ± 2.68	68.07 ± 4.00	55.67 ± 1.11	58.13 ± 1.40	54.72 ± 0.76	55.78 ± 0.87	61.03 ± 1.07
	BART-Base	62.47 ± 5.26	78.10 ± 13.27	84.35 ± 12.71	68.04 ± 8.08	69.60 ± 8.63	65.38 ± 6.44	66.93 ± 7.29	72.11 ± 6.95
	Pretrained Large Language Models								
	GTE-Qwen-1.5B-Instruct	81.24 ± 3.51	88.65 ± 3.86	90.05 ± 4.01	85.13 ± 3.92	85.60 ± 3.63	83.44 ± 3.78	84.22 ± 3.64	85.08 ± 3.73
	Stella_en_1.5B_v5	69.46 ± 7.23	89.01 ± 5.70	91.69 ± 3.49	75.78 ± 4.27	78.54 ± 4.10	73.47 ± 4.68	75.28 ± 4.33	79.55 ± 5.46
	E5-mistral-7b-instruct	78.85 ± 12.42	94.14 ± 2.61	95.02 ± 2.90	78.76 ± 4.52	79.29 ± 4.90	79.09 ± 6.39	79.25 ± 5.64	86.52 ± 7.13
	SFR-Embedding-Mistral	85.67 ± 4.96	92.35 ± 3.90	93.38 ± 3.61	88.57 ± 4.36	90.02 ± 4.00	87.16 ± 4.59	88.18 ± 4.36	89.06 ± 4.39
	Learning from Log Models								
	PreLog	54.16 ± 4.64	73.87 ± 9.27	80.08 ± 6.26	53.30 ± 4.53	55.26 ± 4.25	53.17 ± 3.82	53.83 ± 3.80	64.94 ± 4.84
	Bifrost (Ours)	**90.10 ± 1.25**	**94.67 ± 1.57**	**95.12 ± 1.45**	**92.40 ± 1.29**	**93.38 ± 1.22**	**91.22 ± 1.23**	**92.00 ± 1.25**	**92.43 ± 1.33**
Thunderbird	*Pretrained Small Language Models*								
	Glove-300d	8.07 ± 2.23	38.08 ± 5.15	54.78 ± 3.76	29.80 ± 3.41	42.67 ± 3.28	19.71 ± 2.73	27.68 ± 2.95	27.19 ± 2.50
	BERT-Base	25.77 ± 8.81	36.85 ± 1.49	42.44 ± 2.92	30.59 ± 3.86	37.16 ± 2.60	28.31 ± 5.84	31.22 ± 4.36	34.84 ± 4.61
	ALBERT-Base	38.43 ± 8.43	52.88 ± 4.37	61.96 ± 3.75	42.99 ± 5.48	50.85 ± 4.55	40.20 ± 6.73	43.67 ± 5.83	48.36 ± 5.96
	BART-Base	44.83 ± 5.08	58.94 ± 6.45	66.67 ± 6.63	46.54 ± 5.14	54.35 ± 5.13	45.61 ± 4.67	48.33 ± 4.62	55.37 ± 4.26
	Pretrained Large Language Models								
	GTE-Qwen-1.5B-Instruct	41.98 ± 5.96	67.47 ± 6.76	84.79 ± 7.48	56.96 ± 3.18	72.35 ± 5.38	49.36 ± 4.00	57.15 ± 4.42	58.29 ± 5.12
	Stella_en_1.5B_v5	52.65 ± 8.62	59.95 ± 9.53	69.08 ± 9.82	56.98 ± 9.45	63.63 ± 9.50	54.59 ± 8.95	57.58 ± 9.13	59.80 ± 8.77
	E5-mistral-7b-instruct	42.13 ± 10.22	70.16 ± 9.36	83.28 ± 5.43	57.29 ± 6.14	68.35 ± 4.96	49.80 ± 7.67	56.17 ± 6.69	59.08 ± 8.14
	SFR-Embedding-Mistral	47.38 ± 6.05	77.98 ± 7.70	91.33 ± 3.67	61.34 ± 5.63	72.84 ± 4.84	53.97 ± 5.81	60.45 ± 5.37	64.27 ± 5.54
	Learning from Log Models								
	PreLog	52.93 ± 12.22	74.30 ± 8.04	77.74 ± 7.71	61.66 ± 6.89	68.44 ± 7.26	57.47 ± 8.85	61.29 ± 7.68	63.40 ± 9.29
	Bifrost (Ours)	**75.77 ± 5.16**	**83.85 ± 5.51**	**86.07 ± 5.51**	**81.41 ± 5.31**	**83.18 ± 5.43**	**78.80 ± 5.29**	**80.37 ± 5.31**	**80.32 ± 5.31**
Platform-X	*Pretrained Small Language Models*								
	Glove-300d	52.44 ± 3.17	70.24 ± 0.98	78.21 ± 1.83	56.29 ± 1.69	60.36 ± 1.45	54.22 ± 2.22	56.29 ± 1.55	62.64 ± 1.86
	BERT-Base	72.75 ± 0.77	79.05 ± 0.80	79.68 ± 0.70	71.43 ± 0.91	70.69 ± 0.78	72.05 ± 0.81	71.56 ± 0.76	75.91 ± 0.78
	ALBERT-Base	72.41 ± 1.84	81.42 ± 0.45	82.69 ± 0.21	72.70 ± 1.29	73.49 ± 1.39	72.64 ± 1.54	72.90 ± 1.44	76.98 ± 1.13
	BART-Base	74.80 ± 1.32	79.17 ± 1.16	80.91 ± 0.93	73.52 ± 0.95	74.62 ± 0.92	74.00 ± 1.06	74.11 ± 0.97	77.27 ± 1.04
	Pretrained Large Language Models								
	GTE-Qwen-1.5B-Instruct	83.32 ± 0.37	84.91 ± 0.18	85.24 ± 0.34	80.37 ± 0.35	80.97 ± 0.73	81.78 ± 0.33	81.37 ± 0.36	84.19 ± 0.14
	Stella_en_1.5B_v5	83.15 ± 0.20	84.68 ± 0.59	85.67 ± 0.49	80.01 ± 0.80	80.35 ± 0.98	81.36 ± 0.66	80.91 ± 0.73	84.10 ± 0.33
	E5-mistral-7b-instruct	84.89 ± 0.76	86.29 ± 0.11	86.33 ± 0.08	79.74 ± 0.99	78.63 ± 1.05	82.13 ± 0.93	80.78 ± 0.64	85.57 ± 0.45
	SFR-Embedding-Mistral	84.69 ± 0.98	86.17 ± 0.36	86.26 ± 0.28	80.61 ± 1.05	79.95 ± 1.28	82.50 ± 0.64	81.49 ± 0.58	85.43 ± 0.65
	Learning from Log Models								
	PreLog	69.91 ± 1.61	77.20 ± 2.08	78.02 ± 2.06	71.95 ± 1.83	72.41 ± 2.04	71.11 ± 1.67	71.56 ± 1.78	73.62 ± 1.86
	Bifrost (Ours)	**87.98 ± 0.96**	**92.35 ± 0.86**	**93.08 ± 0.94**	**88.36 ± 0.95**	**88.40 ± 1.42**	**88.33 ± 0.85**	**88.35 ± 1.00**	**90.14 ± 0.93**

Table 3. Comparison results with baseline methods for anomaly detection.

Embedding Model	Parameters	BGL			Hadoop			Thunderbird			Platform-X		
		Precision	Recall	F1-Score	Precision	Recall	F1-Score	Precision	Recall	F1-Score	Precision	Recall	F1-Score
Pretrained Small Language Models													
Glove-300d	120M	88.41 ± 3.54	92.06 ± 0.33	90.16 ± 1.84	85.11 ± 4.31	94.82 ± 0.43	89.65 ± 2.51	84.67 ± 4.22	70.15 ± 13.98	76.13 ± 10.69	88.20 ± 2.73	86.73 ± 2.85	87.37 ± 0.41
BERT-Base	110M	93.90 ± 1.42	89.96 ± 1.33	91.89 ± 1.36	89.04 ± 9.11	94.92 ± 0.37	91.65 ± 5.38	91.50 ± 2.08	81.19 ± 0.37	86.03 ± 0.92	88.29 ± 2.07	92.78 ± 0.16	90.47 ± 1.07
ALBERT-Base	12M	94.60 ± 0.61	92.34 ± 0.08	93.46 ± 0.29	50.87 ± 3.33	94.65 ± 0.14	66.11 ± 2.74	97.25 ± 0.78	77.88 ± 2.38	86.46 ± 1.26	90.28 ± 2.20	87.26 ± 1.04	88.71 ± 0.73
BART-Base	140M	90.53 ± 4.11	91.96 ± 1.22	91.18 ± 1.88	70.18 ± 8.94	94.56 ± 0.39	80.27 ± 5.49	92.60 ± 3.82	79.97 ± 1.93	85.74 ± 1.26	81.37 ± 2.39	84.44 ± 2.35	82.83 ± 1.37
Pretrained Large Language Models													
GTE-Qwen-1.5B-Instruct	1500M	91.85 ± 3.52	93.47 ± 1.45	92.63 ± 2.36	60.40 ± 12.50	94.94 ± 0.30	73.13 ± 8.53	96.94 ± 1.03	80.79 ± 0.77	88.13 ± 0.67	98.72 ± 0.36	90.54 ± 0.22	94.45 ± 0.16
Stella_en_1.5B_v5	1500M	90.69 ± 2.64	90.73 ± 2.92	90.71 ± 2.68	63.99 ± 9.16	95.15 ± 0.29	76.14 ± 6.52	96.53 ± 1.97	83.20 ± 1.45	89.35 ± 0.80	98.92 ± 0.72	90.56 ± 0.17	94.55 ± 0.28
E5-mistral-7b-instruct	7200M	98.45 ± 0.94	94.16 ± 0.90	96.25 ± 0.53	78.84 ± 10.00	95.32 ± 0.52	85.93 ± 6.02	98.75 ± 0.42	84.18 ± 2.39	90.86 ± 1.35	99.13 ± 0.39	91.05 ± 0.20	94.92 ± 0.12
SFR-Embedding-Mistral	7200M	98.65 ± 0.34	95.72 ± 2.16	97.15 ± 0.99	83.12 ± 13.88	95.30 ± 0.66	88.13 ± 8.42	98.59 ± 0.37	81.27 ± 9.32	88.81 ± 5.65	99.08 ± 0.61	90.95 ± 0.25	94.84 ± 0.24
Learning from Log Models													
PreLog	140M	95.94 ± 2.31	89.38 ± 1.20	92.52 ± 0.91	72.72 ± 12.24	96.52 ± 0.13	82.38 ± 7.82	79.17 ± 11.60	75.98 ± 7.43	77.14 ± 8.11	88.34 ± 5.09	82.78 ± 5.90	85.12 ± 1.50
Bifrost (Ours)	140M	**98.81 ± 0.67**	**98.34 ± 0.53**	**98.57 ± 0.52**	**97.56 ± 0.81**	**96.14 ± 0.60**	**96.84 ± 0.22**	**97.50 ± 2.09**	**90.78 ± 2.13**	**94.01 ± 1.72**	**96.33 ± 0.76**	**97.25 ± 0.95**	**96.78 ± 0.21**

task, We evaluate on Platform-X only, since public datasets do not provide fault-type labels, using Macro-Precision, Macro-Recall, and Macro-F1 [5, 39, 59].

5.1.4 ***Implementation Details.*** **BIFROST Training.** BIFROST is built on BART-Base [23] and trained with the proposed CARL strategy using AdamW. We set $(\lambda_1,\lambda_2,\lambda_3,\lambda_4) = (1,0.5,1,1)$ for the joint loss, and set the system failure corpus size to 10 and the system component corpus size to 8 for fallibility learning.

Fault Diagnosis Evaluation. For AD/RCL/FI, we use LogRobust [57], LogRCA [44], and CloudLog [59] as backbones, respectively. Following prior work [20, 31], we apply Drain parsing [10], group logs with a sliding window of 20, and split data into train/val/test with a 6:1:3 ratio. All results are averaged over five runs (mean±std). Experiments are conducted on a Linux server with an NVIDIA H20 GPU. For all baselines, we use their public implementations and default settings [20]. Code is available at https://anonymous.4open.science/r/Bifrost.

Table 4. Comparison results with baseline methods for fault identification.

Embedding Model	Macro-Precision	Macro-Recall	Macro-F1-Score
Pretrained Small Language Models			
Glove-300d	80.95 ± 0.50	77.29 ± 1.95	77.26 ± 1.13
BERT-Base	77.68 ± 2.65	72.11 ± 3.28	72.92 ± 3.15
ALBERT-Base	73.62 ± 8.14	71.44 ± 4.59	70.76 ± 6.15
BART-Base	47.47 ± 5.94	66.15 ± 1.92	54.86 ± 4.27
Pretrained Large Language Models			
GTE-Qwen-1.5B-Instruct	97.66 ± 0.65	88.02 ± 0.63	92.45 ± 0.56
Stella_en_1.5B_v5	96.77 ± 1.69	86.40 ± 3.11	90.76 ± 2.94
E5-mistral-7b-instruct	97.80 ± 1.27	88.25 ± 1.63	92.43 ± 1.76
SFR-Embedding-Mistral	97.78 ± 0.56	88.55 ± 0.61	92.73 ± 0.64
Learning from Log Models			
PreLog	51.00 ± 8.46	65.02 ± 1.01	56.38 ± 4.74
Bifrost (Ours)	**94.84 ± 0.44**	**92.81 ± 0.53**	**93.35 ± 0.43**

5.2 RQ1: How effective is fallibility representation in the log-based fault diagnosis?

This section evaluates the effectiveness of the fallibility representation in log-based fault diagnosis tasks; Tables 2, 3, and 4 present the comparison results with baseline models on three tasks. Overall, Bifrost achieves state-of-the-art log representations. For the AD task, the results obtained by Bifrost are, on average, 10.42%, 6.80%, and 12.26% higher in F1-score than the three baseline methods. For the RCL task, Bifrost outperforms the three baselines by an average of 27.61%, 9.35%, and 17.87% on HR@k. For the FI task, Bifrost achieves an even more significant advantage, surpassing the three baselines by an average of 24.40%, 1.26%, and 36.97% in Macro-F1. This substantial leap indicates that the fallibility representation can indeed capture the rich system failure information embedded in logs. This evidence demonstrates the potent effectiveness of the fallibility representation in fault diagnosis tasks.

Table 5. Ablation results on the root cause localization task.

Dataset	Learning Objective				Metrics							
	CARL	EFP	AED	SCP	HR@1	HR@3	HR@5	PR@3	PR@5	MAP@3	MAP@5	MRR
BGL	✓	✓	✗	✗	82.85 ± 3.59	88.86 ± 3.74	89.69 ± 4.00	85.55 ± 3.47	86.23 ± 3.16	84.23 ± 3.44	85.02 ± 3.38	85.81 ± 3.66
	✓	✗	✓	✗	81.24 ± 3.51	88.65 ± 3.86	90.05 ± 4.01	85.13 ± 3.92	85.60 ± 3.63	83.44 ± 3.78	84.22 ± 3.64	85.08 ± 3.73
	✓	✗	✗	✓	83.05 ± 3.99	89.05 ± 3.41	90.54 ± 3.25	81.21 ± 9.94	82.48 ± 10.9	81.70 ± 7.39	81.89 ± 8.74	86.20 ± 3.54
	✓	✗	✓	✓	85.98 ± 3.40	91.29 ± 2.99	92.08 ± 2.74	88.39 ± 3.14	89.34 ± 3.02	87.24 ± 3.22	88.01 ± 3.15	88.80 ± 3.09
	✓	✓	✗	✓	86.52 ± 1.98	91.86 ± 1.13	92.42 ± 1.05	89.34 ± 1.15	90.22 ± 1.06	88.05 ± 1.46	88.85 ± 1.30	89.35 ± 1.43
	✓	✓	✓	✗	87.86 ± 1.11	92.57 ± 1.33	93.31 ± 1.52	90.01 ± 1.11	91.03 ± 1.26	88.96 ± 1.12	89.70 ± 1.14	90.44 ± 1.18
	✗	✓	✓	✓	83.92 ± 1.79	90.27 ± 1.90	91.39 ± 1.76	87.55 ± 1.77	88.91 ± 1.72	85.80 ± 1.71	86.95 ± 1.71	87.32 ± 1.56
	✓	✓	✓	✓	**90.10 ± 1.25**	**94.67 ± 1.57**	**95.12 ± 1.45**	**92.40 ± 1.29**	**93.38 ± 1.22**	**91.22 ± 1.23**	**92.00 ± 1.25**	**92.43 ± 1.33**
Thunderbird	✓	✓	✗	✗	66.79 ± 7.61	76.08 ± 6.58	77.70 ± 6.71	73.02 ± 7.01	75.14 ± 6.75	70.09 ± 7.24	71.94 ± 7.08	71.80 ± 7.14
	✓	✗	✓	✗	67.11 ± 3.77	74.66 ± 4.02	76.75 ± 3.70	72.21 ± 4.08	73.80 ± 3.95	69.88 ± 4.03	71.30 ± 4.01	71.21 ± 3.80
	✓	✗	✗	✓	65.39 ± 5.92	75.45 ± 7.08	78.28 ± 7.38	72.13 ± 6.72	74.74 ± 7.09	68.86 ± 6.34	70.98 ± 6.61	70.87 ± 6.34
	✓	✗	✓	✓	70.31 ± 4.06	79.23 ± 3.62	81.69 ± 3.94	76.00 ± 3.86	78.29 ± 3.83	73.26 ± 3.93	75.06 ± 3.82	75.23 ± 3.72
	✓	✓	✗	✓	68.99 ± 9.46	78.49 ± 8.54	81.13 ± 8.58	74.94 ± 8.45	77.49 ± 8.46	72.01 ± 8.83	73.96 ± 8.68	74.16 ± 8.89
	✓	✓	✓	✗	71.70 ± 4.35	81.49 ± 6.27	84.08 ± 6.60	78.49 ± 5.44	80.83 ± 6.04	75.37 ± 4.90	77.34 ± 5.32	76.98 ± 5.08
	✗	✓	✓	✓	70.67 ± 2.84	83.65 ± 3.86	85.72 ± 3.58	78.81 ± 2.91	81.67 ± 3.25	74.90 ± 2.82	77.36 ± 2.88	77.34 ± 2.98
	✓	✓	✓	✓	**75.77 ± 5.16**	**83.85 ± 5.51**	**86.07 ± 5.51**	**81.41 ± 5.31**	**83.18 ± 5.43**	**78.80 ± 5.29**	**80.37 ± 5.31**	**80.32 ± 5.31**
Platform-X	✓	✓	✗	✗	82.43 ± 1.58	88.55 ± 1.86	90.49 ± 1.93	81.57 ± 1.90	80.45 ± 2.17	82.18 ± 1.73	81.60 ± 1.85	85.62 ± 1.69
	✓	✗	✓	✗	82.52 ± 0.71	89.05 ± 0.97	91.45 ± 1.13	79.43 ± 2.04	77.73 ± 2.31	81.19 ± 1.38	79.94 ± 1.75	86.12 ± 0.76
	✓	✗	✗	✓	81.98 ± 1.15	87.79 ± 2.05	89.66 ± 2.57	80.38 ± 1.99	79.09 ± 2.50	81.30 ± 1.22	80.51 ± 1.55	85.09 ± 1.65
	✓	✗	✓	✓	85.61 ± 1.91	90.89 ± 1.18	91.95 ± 0.93	85.57 ± 2.55	84.57 ± 3.74	85.81 ± 2.18	85.39 ± 2.68	88.27 ± 1.45
	✓	✓	✗	✓	84.07 ± 1.60	89.22 ± 1.47	90.36 ± 1.33	83.94 ± 2.77	83.04 ± 3.87	84.16 ± 2.30	83.78 ± 2.82	86.68 ± 1.41
	✓	✓	✓	✗	86.64 ± 2.52	92.32 ± 1.49	93.78 ± 1.04	84.95 ± 2.38	84.16 ± 2.70	85.93 ± 2.36	85.27 ± 2.45	89.60 ± 1.90
	✗	✓	✓	✓	86.55 ± 0.24	91.98 ± 0.65	92.86 ± 0.64	85.60 ± 0.68	84.64 ± 1.81	86.29 ± 0.44	85.74 ± 0.86	89.27 ± 0.31
	✓	✓	✓	✓	**87.98 ± 0.96**	**92.35 ± 0.86**	**93.08 ± 0.94**	**88.36 ± 0.95**	**88.40 ± 1.42**	**88.33 ± 0.85**	**88.35 ± 1.00**	**90.14 ± 0.93**

5.3 RQ2: What are the contributions of different training strategies to Bifrost?

This section conducts an ablation study to evaluate the effectiveness of the three learning objectives and one learning strategy in Bifrost. The results of the ablation study are shown in Table 5, 6 and 7. Overall, the ablation of all training strategies suffered varying degrees of performance

Table 6. Ablation results on the anomaly detection task.

Learning Objective				BGL			Hadoop			Thunderbird			Platform-X		
CARL	EFP	AED	SCP	Precision	Recall	F1-Score	Precision	Recall	F1-Score	Precision	Recall	F1-Score	Precision	Recall	F1-Score
✓	✓	✗	✗	98.45 ± 0.52	92.52 ± 1.86	95.39 ± 1.12	95.73 ± 1.43	95.55 ± 0.55	95.63 ± 0.78	96.36 ± 1.09	86.31 ± 4.06	91.03 ± 2.64	92.35 ± 2.09	94.46 ± 0.75	93.38 ± 1.19
✓	✗	✓	✗	98.11 ± 0.83	92.95 ± 1.59	95.45 ± 0.93	96.38 ± 1.69	95.49 ± 0.44	95.93 ± 0.83	97.89 ± 1.34	82.83 ± 4.77	89.63 ± 2.30	94.33 ± 1.77	89.99 ± 0.68	92.10 ± 0.58
✓	✗	✗	✓	98.10 ± 1.70	88.49 ± 1.18	93.03 ± 0.36	82.29 ± 5.84	97.34 ± 0.24	89.07 ± 3.44	96.15 ± 0.97	80.95 ± 0.50	87.89 ± 0.63	96.15 ± 1.24	89.54 ± 0.51	92.72 ± 0.48
✓	✗	✓	✓	98.66 ± 0.44	93.53 ± 2.27	96.01 ± 1.06	96.44 ± 0.54	96.00 ± 0.77	96.22 ± 0.40	96.45 ± 1.01	85.77 ± 2.45	90.78 ± 1.57	94.20 ± 2.50	97.23 ± 0.16	95.68 ± 1.27
✓	✓	✗	✓	98.70 ± 0.73	96.06 ± 2.08	97.35 ± 0.99	97.66 ± 0.70	95.87 ± 0.65	96.75 ± 0.30	94.77 ± 3.62	89.11 ± 2.80	91.78 ± 1.90	94.54 ± 1.14	95.92 ± 0.62	95.22 ± 0.35
✓	✓	✓	✗	99.15 ± 0.43	97.39 ± 1.53	98.25 ± 0.64	97.51 ± 0.98	95.72 ± 0.76	96.60 ± 0.49	96.84 ± 1.43	89.01 ± 2.23	92.74 ± 1.43	92.67 ± 2.06	94.95 ± 0.65	93.78 ± 0.91
✗	✓	✓	✓	98.84 ± 0.71	95.37 ± 3.25	97.04 ± 1.76	96.33 ± 0.26	94.34 ± 2.58	95.14 ± 1.58	95.18 ± 1.69	84.60 ± 2.18	89.55 ± 1.27	93.23 ± 1.55	96.01 ± 0.73	94.59 ± 0.69
✓	✓	✓	✓	**98.81 ± 0.67**	**98.34 ± 0.53**	**98.57 ± 0.52**	**97.56 ± 0.81**	**96.14 ± 0.60**	**96.84 ± 0.22**	**97.50 ± 2.09**	**90.78 ± 2.13**	**94.01 ± 1.72**	**96.33 ± 0.76**	**97.25 ± 0.95**	**96.78 ± 0.21**

Table 7. Ablation results on the fault identification task.

Learning Objective				Platform-X		
CARL	EFP	AED	SCP	Macro-Precision	Macro-Recall	Macro-F1
✓	✓	✗	✗	81.63 ± 1.25	78.62 ± 1.19	77.89 ± 1.09
✓	✗	✓	✗	82.57 ± 1.66	78.93 ± 1.91	77.82 ± 1.79
✓	✗	✗	✓	98.17 ± 0.46	91.80 ± 1.89	94.48 ± 0.89
✓	✗	✓	✓	93.00 ± 0.82	90.67 ± 1.42	90.80 ± 1.65
✓	✓	✗	✓	93.06 ± 0.53	91.20 ± 0.37	91.52 ± 0.49
✓	✓	✓	✗	83.18 ± 1.34	81.93 ± 1.73	80.90 ± 1.79
✗	✓	✓	✓	86.95 ± 2.38	85.68 ± 1.97	85.54 ± 2.17
✓	✓	✓	✓	**94.84 ± 0.44**	**92.81 ± 0.53**	**93.35 ± 0.43**

degradation, which demonstrates the effectiveness of the proposed strategies. In the ablation of the three learning objectives, ablating only a single objective caused an average performance drop of 1.45%, 2.66%, and 5.61% on the three tasks. Simultaneously ablating multiple learning objectives showed a more significant performance drop, with an average of 3.95%, 6.03%, and 9.95%. This indicates that the three well-designed learning objectives indeed help in learning the fallibility representations of logs. Furthermore, the ablation of the CARL strategy also showed a significant performance degradation, causing an average performance loss of 2.47%, 2.44%, and 7.81% on the three tasks. This evidence proves that the CARL strategy can indeed alleviate the fallibility jitter problem, thereby enabling the learning of higher-quality log representations.

5.4 RQ3: How sensitive is BIFROST to the learning of fallibility representations?

This section examines the sensitivity of BIFROST to the supervision proxy used in fallibility representation learning. We vary the size K of the system failure corpus, which provides failure phrases for contrastive negative synthesis, and keep other settings unchanged. Figure 4 reports the results. Overall, BIFROST is robust to a wide range of K. Across datasets, AD fluctuates mildly, and its average span is no more than 2.16%. FI is also stable within a narrow band on Platform-X, and its span is about 1.08%. By contrast, RCL shows higher sensitivity, and its average span is about 5.64%, which suggests that root-cause ranking benefits more from richer failure cues. Performance gains saturate when K increases beyond the default setting. When K is overly small, performance consistently degrades. For example, when K decreases from 10 to 4, AD drops by 0.71% to 2.01% across datasets, and RCL drops by 3.56% to 5.23%. These results indicate that BIFROST does not rely on a finely tuned corpus size. It instead achieves stable fallibility learning, and it exhibits diminishing returns as more failure phrases are introduced.

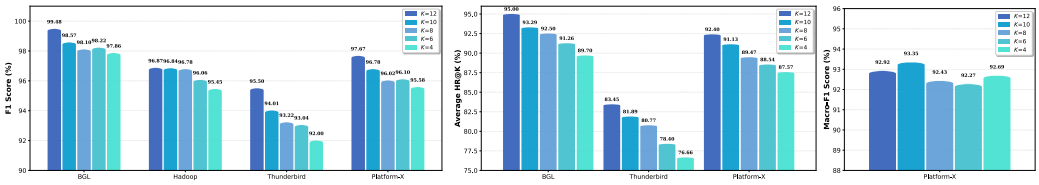
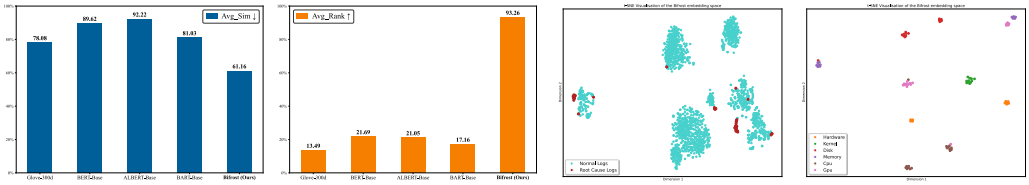


Fig. 4. Parameter sensitivity analysis of fallibility representation learning with respect to the size of the system failure corpus. From left to right, the panels report results on AD, RCL, and FI, respectively.



(a) (b) (c) (d)

Fig. 5. Evaluation of Bifrost's capability to encode fallibility representations. Fig. 5a and Fig. 5b evaluate the representational similarity between expected and unexpected events produced by Bifrost. Fig. 5c shows the event-level representations generated by Bifrost, and Fig. 5c shows the component-level representations.

5.5 RQ4: How capable is Bifrost of encoding the fallibility representation?

This section evaluates whether Bifrost effectively encodes the fallibility representations of logs. To this end, we examined the log representations of Bifrost, following the experimental setup in Section 3. Specifically, at the execution flow level, as shown in Figure 5a and Figure 5b, Bifrost achieved better performance on both the *Avg_Sim* and *Avg_Rank* metrics, indicating its ability to identify unexpected execution flows. In Figure 5c, at the event level, compared to baseline PLMs, the anomalous event representations encoded by Bifrost aggregate into distinct small clusters, demonstrating significant separability. In Figure 5d, at the component level, the individual components in the log representations encoded by Bifrost cluster together respectively, which showcases its awareness of system components. This evidence demonstrates that Bifrost can indeed effectively learn the fallibility representations of logs through its well-designed training strategies, thereby achieving effective fault diagnosis.

6 Threats to Validity

Method. Our approach learns fallibility representations by contrastive training on injected counterfactual failures. Injection can introduce spurious cues and reduce generalizability. We mitigate this with randomized injection and self supervised training. Cross system results and ablations suggest the representations transfer beyond injected artifacts, but injection is an approximation.

Evaluation. Our evaluation is constrained by dataset availability and label granularity. For AD, we use BGL, Thunderbird, Hadoop, and Platform-X. For RCL, we exclude Hadoop due to missing root cause log labels. For FI, we use only Platform-X because most FI datasets rely on metrics and traces, while Platform-X supports component level attribution from logs. This choice may limit transfer to broader benchmarks, we mitigate by using heterogeneous datasets for the first two tasks. Fault identification under alternative labeling schemes remains future work.

7 Related Work

7.1 Log-based Software System Fault Diagnosis

Discovering and identifying problems by analyzing logs has always been an active area of research [12, 37, 48, 54, 59]. Recent fault diagnosis work has been studied from the three tasks of AD, RCL, and FI. Some state-of-the-art AD methods [4, 50, 57] first extract execution flow sequences, then learn the patterns of historical execution flow sequences and compare them with log sequences in the production environment to detect conflicts. Other works focus on localizing the root causes of anomalies. LogRCA [44] ranks all logs within the investigation time window based on PU-Learning. Onion [58] proposed a form of log aggregation called Log Clique to identify event-indicating logs. Other works [5, 39, 59] have studied the classification of faults, mainly by constructing deep learning-based models to capture the sequential features of faults. However, all these works use PLMs as the log representation method, which cannot bridge the structural gap between logs and natural

language, thus failing to meet the requirements of fault diagnosis tasks. BIFROST addresses this by compelling PLMs to learn fallibility representations of logs through meticulously designed tasks.

7.2 System Log Representation

Learning log-oriented representations from system logs has been extensively studied in recent years [6, 8, 21, 29, 47]. LLMeLog [8] conducted an empirical study whose results show that log representations encoded by existing PLMs suffer from issues such as content missing, which it improves by rewriting system logs using LLMs. KnowLog [29] proposed a training framework enhanced by system knowledge, which learns the abbreviation, context, and style knowledge of logs from system logs through well-designed training tasks to improve the log representation. PreLog [21] is a PLM pre-trained on system logs, designed to uniformly perform various log analysis tasks through the paradigm of prompt-tuning. For AD tasks, LogFormer [6] proposed a cross-system anomaly detection paradigm, which aims to learn general system representations and adapt to different systems through fine-tuning. Although these works all learn from system logs, they only focus on the semantic information of logs, ignoring their multi-level structural information, and are not designed for fault diagnosis tasks. In contrast, BIFROST draws inspiration from the log analysis experience of SREs, compelling PLMs to learn fallibility representations of logs to obtain high-quality log representations oriented toward fault diagnosis tasks.

8 Conclusion

In this paper, we propose BIFROST, a log representation learning method for log-based fault diagnosis. Inspired by the log analysis experience of SREs, BIFROST utilizes multi-level information from logs—such as execution flows, events, and components—as fallibility representations and learns them through meticulously designed strategies based on self-supervised contrastive learning to enhance the capabilities of PLMs in fault diagnosis. Comprehensive evaluations on three public systems and one industrial MLaaS system show that the log representations generated by BIFROST outperform existing PLMs by average margins of 9.83% in F1 for AD, 18.28% in HR@k for RCL, and 20.88% in Macro-F1 for FI. This evidence highlights the powerful role of BIFROST and its fallibility representations in fault diagnosis. In the future, we plan to explore larger-scale log representation models based on fallibility representations to achieve more effective and general fault diagnosis.

A System Components Corpus

In the SCP task, component semantics are represented by injecting a component keyword sampled from a predefined set $C = \{c_1, \ldots, c_{|C|}\}$. We instantiate $|C| = 8$ common component types in distributed systems: `cpu`, `gpu`, `disk`, `hardware`, `memory`, `kernel`, `network`, and `file`.

This design is motivated by the practical observation that system failures often manifest at the component granularity. As these component concepts are largely independent of specific applications and log templates, the resulting keyword set provides a portable and transferable inductive bias, enabling better generalization across systems.

B Gradient Analysis for Fallibility Jitter

In this section, we analyze the optimization process of fallibility representation learning from the perspective of gradients in the optimization space. We find that due to the random sampling of system logs and system components during optimization, the gradient trajectory of the optimization process deviates from the ideal gradient trajectory. This is the root cause of the instability in fallibility representation learning, which we term the fallibility jitter problem.

Let $\mathbf{W}$ denote all trainable parameters in BIFROST. At optimization step s, we sample a time index t from the normal log stream $\mathbf{L} = [l_1, \ldots, l_T]$ and l_t. We form the context window $\mathbf{x}_t = [l_{t-w}, \ldots, l_{t-1}]$,

sample a context log $l_i \in \mathbf{x}_t$ for EFP, and sample a component keyword $c_j \in C$ from the component vocabulary C for SCP. Following Sec. 4.2.4, we construct the SCP anchor by injecting c_j into l_t, i.e., $l_{t,j} = l_t \oplus c_j$. We denote the step-wise stochasticity induced by context and component sampling as $\boldsymbol{\xi}_s = (l_i, c_j)$. Making the dependence on $\mathbf{W}$ explicit, the per-step vanilla loss in Eq. 11 is:

$$\mathcal{L}_{\text{Vanilla}}(\mathbf{W}; l_t, \boldsymbol{\xi}_s) = \lambda_1 \mathcal{L}_{\text{EFP}}(\mathbf{W}; \hat{e}_t, l_i^+, \{\tilde{l}_{t,k}\}_{k=1}^K) + \lambda_2 \mathcal{L}_{\text{AED}}(\mathbf{W}; l_t, l_t^+, \{\tilde{l}_{t,k}\}_{k=1}^K) + \lambda_3 \mathcal{L}_{\text{SCP}}(\mathbf{W}; l_{t,j}, l_{t,j}^+, \{\tilde{l}_{t,j,z}\}_{z=1}^Z), \tag{15}$$

The **Stochastic Gradient** of $\mathcal{L}_{\text{Vanilla}}$ is:

$$\mathbf{g}_{\text{Vanilla}}(\mathbf{W}; l_t, \boldsymbol{\xi}) = \nabla_{\mathbf{W}} \mathcal{L}_{\text{Vanilla}}(\mathbf{W}; l_t, \boldsymbol{\xi}), \tag{16}$$

Here, the stochastic gradient is typically considered an unbiased estimator of the **True Gradient**. For $\mathcal{L}_{\text{Vanilla}}$, the true gradient is the gradient obtained by taking the expectation over all sources of randomness, conditioned on l_t:

$$\mathbf{G}_{\text{Vanilla}}(\mathbf{W}; l_t) = \mathbb{E}_{\boldsymbol{\xi}}[\mathbf{g}_{\text{Vanilla}}(\mathbf{W}; l_t, \boldsymbol{\xi})], \tag{17}$$

During the fallibility representation learning process, the deviation between the stochastic gradient and the true gradient is a source of error. We use the **Gradient Variance** to quantify the expected squared norm of this deviation:

$$\sigma^2_{\text{Vanilla}}(\mathbf{W}; l_t) = \mathbb{E}_{\boldsymbol{\xi}}\left[\|\mathbf{g}_{\text{Vanilla}}(\mathbf{W}; l_t, \boldsymbol{\xi}) - \mathbf{G}_{\text{Vanilla}}(\mathbf{W}; l_t)\|^2\right], \tag{18}$$

Since the sampling processes of l_i and c_j are independent and the stochastic terms are separable, we can decompose the total variance. Let $\mathbf{g}_{\text{EFP}}(\mathbf{W}; l_t, l_i) = \nabla_{\mathbf{W}}(\lambda_1 \mathcal{L}_{\text{EFP}})$ and $\mathbf{g}_{\text{SCP}}(\mathbf{W}; l_t, c_j) = \nabla_{\mathbf{W}}(\lambda_3 \mathcal{L}_{\text{SCP}})$. The total variance is the sum of the variances of the gradient terms corresponding to each independent random variable:

$$\sigma^2_{\text{Vanilla}}(\mathbf{W}; l_t) = \text{Var}_{l_i \in \mathbf{x}_t}(\mathbf{g}_{\text{EFP}}(\mathbf{W}; l_t, l_i)) + \text{Var}_{c_j \in C}\left(\mathbf{g}_{\text{SCP}}(\mathbf{W}; l_t, c_j)\right), \tag{19}$$

where for a random variable $\mathbf{X}$, $\text{Var}(\mathbf{X}) = \mathbb{E}[\|\mathbf{X} - \mathbb{E}[\mathbf{X}]\|^2]$. These two variance terms originate from the two random sampling strategies in the fallibility learning process. If s is the training step, η_s is the learning rate, and $\Delta(\cdot,\cdot)$ represents the update direction vector defined by the optimizer, we can define two optimization:

(1) **Ideal Trajectory**: Driven by the $\mathbf{G}_s(\mathbf{W}) \triangleq \mathbf{G}(\mathbf{W}; l_s)$: $\mathbf{W}_{s+1} = \mathbf{W}_s - \eta_s \Delta(\mathbf{W}_s, \mathbf{G}_s)$.
(2) **Stochastic Trajectory**: Driven by the $\mathbf{g}_s(\mathbf{W}) \triangleq \mathbf{g}(\mathbf{W}; l_s, \boldsymbol{\xi}_s)$: $\mathbf{W}'_{s+1} = \mathbf{W}'_s - \eta_s \Delta(\mathbf{W}'_s, \mathbf{g}_s)$.

The error in each optimization step can be defined as the weight error between these two trajectories: $\mathbf{E}_s = \mathbf{W}_s - \mathbf{W}'_s$. This weight error exists in every training step and is the source of instability in fallibility representation learning, which we call fallibility jitter.

C Weight Error Upper Bound of Fallibility Jitter

To further investigate the fallibility jitter problem, we derive the upper bound on the cumulative weight error during the optimization process for two mainstream optimizers, Stochastic Gradient Descent (SGD) and Adam, thereby quantifying the impact of fallibility jitter on the representation space.

C.1 SGD as the Optimizer

For standard SGD, the update vector is $\Delta(\mathbf{W}, \mathbf{g}) \triangleq \mathbf{g}$. The single-step evolution of the weight error is:

$$\mathbf{E}_{s+1} = \mathbf{W}_{s+1} - \mathbf{W}'_{s+1} = (\mathbf{W}_s - \eta_s \mathbf{G}_s(\mathbf{W}_s)) - (\mathbf{W}'_s - \eta_s \mathbf{g}_s(\mathbf{W}'_s)) = \mathbf{E}_s - \eta_s(\mathbf{G}_s(\mathbf{W}_s) - \mathbf{g}_s(\mathbf{W}'_s)), \tag{20}$$

Applying the norm and using the triangle inequality, we get:

$$\begin{aligned} \|\mathbf{E}_{s+1}\| &= \left\|\mathbf{E}_s - \eta_s(\mathbf{G}_s(\mathbf{W}_s) - \mathbf{G}_s(\mathbf{W}'_s) + \mathbf{G}_s(\mathbf{W}'_s) - \mathbf{g}_s(\mathbf{W}'_s))\right\| \\ &\leq \|\mathbf{E}_s\| + \eta_s\left\|\mathbf{G}_s(\mathbf{W}_s) - \mathbf{G}_s(\mathbf{W}'_s)\right\| + \eta_s\left\|\mathbf{g}_s(\mathbf{W}'_s) - \mathbf{G}_s(\mathbf{W}'_s)\right\|, \end{aligned} \tag{21}$$

We assume $\mathbf{G}s(\cdot)$ is LLip-Lipschitz and define $\boldsymbol{\delta}_s(\mathbf{W}) = \mathbf{g}_s(\mathbf{W}) - \mathbf{G}_s(\mathbf{W})$. Thus,

$$\|\mathbf{G}_s(\mathbf{W}_a) - \mathbf{G}_s(\mathbf{W}_b)\| \le L_{\text{Lip}} \|\mathbf{W}_a - \mathbf{W}_b\|, \quad \|\mathbf{E}_{s+1}\| \le (1+\eta_s L_{\text{Lip}})\|\mathbf{E}_s\| + \eta_s \left\|\boldsymbol{\delta}_s(\mathbf{W}'_s)\right\|, \tag{22}$$

Assuming a constant learning rate $\eta_s = \eta$ and unrolling this inequality from $s=0$ to $S-1$:

$$\begin{aligned}
\|\mathbf{E}_S\| &\le (1+\eta L_{\text{Lip}})\|\mathbf{E}_{S-1}\| + \eta \left\|\boldsymbol{\delta}_{S-1}(\mathbf{W}'_{S-1})\right\| \\
&\le (1+\eta L_{\text{Lip}})\left[(1+\eta L_{\text{Lip}})\|\mathbf{E}_{S-2}\| + \eta \left\|\boldsymbol{\delta}_{S-2}(\mathbf{W}'_{S-2})\right\|\right] + \eta \left\|\boldsymbol{\delta}_{S-1}(\mathbf{W}'_{S-1})\right\| \\
&\le \sum_{s=0}^{S-1} \eta (1+\eta L_{\text{Lip}})^{S-1-s} \left\|\boldsymbol{\delta}_s(\mathbf{W}'_s)\right\|,
\end{aligned} \tag{23}$$

Taking the expectation of both sides with respect to all sources of randomness ($\{\boldsymbol{\xi}_s\}_{s=0}^{S-1}$) and applying Jensen's inequality:

$$\mathbb{E}[\|\mathbf{E}_S\|] \le \eta \sum_{s=0}^{S-1} (1+\eta L_{\text{Lip}})^{S-1-s} \mathbb{E}[\left\|\boldsymbol{\delta}_s(\mathbf{W}'_s)\right\|] \le \eta \sum_{s=0}^{S-1} (1+\eta L_{\text{Lip}})^{S-1-s} \sqrt{\mathbb{E}[\|\boldsymbol{\delta}_s(\mathbf{W}'_s)\|^2]}, \tag{24}$$

Since $\mathbb{E}_{\boldsymbol{\xi}_s}[\left\|\boldsymbol{\delta}_s(\mathbf{W}'_s)\right\|^2 | \mathbf{W}'_s] = \sigma^2(\mathbf{W}'_s; l_s)$, we obtain the final upper bound on the weight error:

$$\mathbb{E}[\|\mathbf{E}_S\|] \le \eta \sum_{s=0}^{S-1} (1+\eta L_{\text{Lip}})^{S-1-s} \mathbb{E}_{\mathbf{W}'_s}[\sigma(\mathbf{W}'_s; l_s)], \tag{25}$$

This equation shows that the gradient standard deviation $\sigma(\mathbf{W}'_s; l_s)$ at each step is amplified and accumulated into the final weight error, thereby affecting the fallibility representation learning.

C.2 Adam as the Optimizer

The Adam optimizer's update depends on the exponential moving average estimates of the first moment $\mathbf{m}_s$ and the second moment $\mathbf{v}_s$ of the gradients. The gradient noise contaminates both of them.

C.2.1 ***The First Moment Estimate.*** The update rules for the ideal and stochastic momentum:

$$\mathbf{m}_{s+1} = \beta_1 \mathbf{m}_s + (1-\beta_1)\mathbf{G}_s(\mathbf{W}_s), \quad \mathbf{m}'_{s+1} = \beta_1 \mathbf{m}'_s + (1-\beta_1)\mathbf{g}_s(\mathbf{W}'_s), \tag{26}$$

Define the momentum deviation as $\Delta\mathbf{m}_s \triangleq \mathbf{m}_s - \mathbf{m}'_s$.

$$\begin{aligned}
\|\Delta\mathbf{m}_{s+1}\| &= \left\|\beta_1 \Delta\mathbf{m}_s + (1-\beta_1)(\mathbf{G}_s(\mathbf{W}_s) - \mathbf{g}_s(\mathbf{W}'_s))\right\| \\
&\le \beta_1 \|\Delta\mathbf{m}_s\| + (1-\beta_1)\left(\left\|\mathbf{G}_s(\mathbf{W}_s) - \mathbf{G}_s(\mathbf{W}'_s)\right\| + \left\|\boldsymbol{\delta}_s(\mathbf{W}'_s)\right\|\right) \\
&\le \beta_1 \|\Delta\mathbf{m}_s\| + (1-\beta_1)(L_{\text{Lip}}\|\mathbf{E}_s\| + \left\|\boldsymbol{\delta}_s(\mathbf{W}'_s)\right\|),
\end{aligned} \tag{27}$$

This recursive relationship shows that the gradient noise $\|\boldsymbol{\delta}_s\|$ directly drives the accumulation of momentum deviation, and its influence decays over time with a factor of β_1.

C.2.2 ***The Second Moment Estimate.*** The update of the stochastic second moment $\mathbf{v}'_s$ depends on $\mathbf{g}_s(\mathbf{W}'_s)^2$. Its expectation is:

$$\begin{aligned}
&\mathbb{E}_{\boldsymbol{\xi}_s}[\mathbf{g}_s(\mathbf{W}'_s)^2] = \mathbb{E}[(\mathbf{G}_s(\mathbf{W}'_s) + \boldsymbol{\delta}_s(\mathbf{W}'_s))^2] \\
&= \mathbf{G}_s(\mathbf{W}'_s)^2 + 2\mathbf{G}_s(\mathbf{W}'_s) \odot \mathbb{E}[\boldsymbol{\delta}_s(\mathbf{W}'_s)] + \mathbb{E}[\boldsymbol{\delta}_s(\mathbf{W}'_s)^2] = \mathbf{G}_s(\mathbf{W}'_s)^2 + \text{diag}(\text{Var}(\mathbf{g}_s)),
\end{aligned} \tag{28}$$

where $\mathbb{E}[\boldsymbol{\delta}_s] = \mathbf{0}$, and $\mathbb{E}[\delta_{s,i}^2]$ is the variance of the i-th component of the gradient. This implies that $\mathbf{g}_s^2$ is a biased estimator of $\mathbf{G}_s^2$, with bias determined by the gradient variance. Through exponential averaging, this bias propagates into $\mathbf{v}'_s$, distorting the learning rate $\eta/(\sqrt{\hat{\mathbf{v}}'_s} + \epsilon)$ across parameters.

D Weight Error Upper Bound Analysis for CARL

In this section, we demonstrate that the stochastic gradient $\mathbf{g}_{\text{CARL}}$ of the CARL strategy has lower variance, thereby mitigating fallibility jitter and enabling robust fallibility representation learning. This is achieved through two mechanisms: variance isolation and negative covariance constraint.

D.1 Variance Isolation

In CARL, the randomness introduced by context sampling l_i is isolated into an independent loss term, $\mathcal{L}_{\text{CL}}$, with a weight of λ_2. In contrast, in the vanilla strategy, the entire EFP task, with a weight of λ_1, depends on l_i. Assuming that the gradient norms are of a comparable magnitude in both strategies, as long as λ_2 in CARL is smaller than λ_1 in the vanilla strategy, the contribution to the gradient variance from the sampling of l_i will be reduced:

$$\text{Var}_{l_i}(\nabla_{\mathbf{W}}(\lambda_2\mathcal{L}_{\text{CL}})) \propto \lambda_2^2 \cdot \text{Var}_{l_i}(\nabla_{\mathbf{W}}\mathcal{L}_{\text{CL}}) < \lambda_1^2 \cdot \text{Var}_{l_i}(\nabla_{\mathbf{W}}\mathcal{L}_{\text{EFP}}), \tag{29}$$

D.2 Negative Covariance Constraint

In the CARL strategy, when an anchor $l_{t,j} = l_t \oplus c_j$ is sampled for the SCP task, the remaining component-injected logs $\{l_{t,i}\}_{i\neq j}$ are shared negatives for other loss terms. For a sampled component c_j, the stochastic gradient part decomposes as $\mathbf{G}(c_j) = \mathbf{A}(c_j) + \mathbf{B}(c_j)$:

- **Primary Term** $\mathbf{A}(c_j)$: Originates from using $l_{t,j}$ as the anchor for the SCP task.

$$\mathbf{A}(c_j) \triangleq \nabla_{\mathbf{W}}(\lambda_4\mathcal{L}_{\text{SCP}}(\text{anchor} = l_{t,j}, \ldots)), \tag{30}$$

- **Covariate Term** $\mathbf{B}(c_j)$: Originates from using the non-anchor injected logs $\{l_{t,i}\}_{i\neq j}$ as shared negative samples for other tasks.

$$\mathbf{B}(c_j) \triangleq \nabla_{\mathbf{W}} \sum_{\text{task}\in\{\text{EFP,CL,AED}\}} \lambda_{\text{task}}\mathcal{L}_{\text{task}}(\ldots, \text{Negatives} = \{l_{t,i}\}_{i\neq j}), \tag{31}$$

$$\text{Var}(\mathbf{G}) = \text{Var}(\mathbf{A}) + \text{Var}(\mathbf{B}) + 2\text{Cov}(\mathbf{A},\mathbf{B}), \tag{32}$$

If $\text{Cov}(\mathbf{A},\mathbf{B}) < 0$, this strategy reduces the gradient variance. We define the gradient vectors

- $\mathbf{g}_{A,i}$: The gradient generated when c_i is selected as the SCP anchor. $\mathbf{A}(c_i) \triangleq \mathbf{g}_{A,i}$.
- $\mathbf{g}_{B,i}$: The gradient generated when $l_{t,i}$ is used as a shared negative sample. $\mathbf{B}(c_j) \triangleq \sum_{i\neq j}\mathbf{g}_{B,i}$.

Since the objectives of both $\mathbf{g}_{A,i}$ and $\mathbf{g}_{B,i}$ are to push away $l_{t,i}$, their gradient directions are highly aligned. Thus, $\{\mathbf{g}_{A,i}\}_{i=1}^N$ and $\{\mathbf{g}_{B,i}\}_{i=1}^N$ are strongly positively correlated.

$$\text{Cov}_i(\mathbf{g}_{A,i}, \mathbf{g}_{B,i}) = \mathbb{E}_i[\mathbf{g}_{A,i}^T\mathbf{g}_{B,i}] - \mathbb{E}_i[\mathbf{g}_{A,i}]^T\mathbb{E}_i[\mathbf{g}_{B,i}] > 0, \tag{33}$$

Let $|C| = N$ and assume the sampling is uniform. The expectation of the cross-term is:

$$\mathbb{E}[\mathbf{A}] = \mathbb{E}_j[\mathbf{g}_{A,j}] = \tfrac{1}{N}\textstyle\sum_{i=1}^N \mathbf{g}_{A,i} \triangleq \bar{\mathbf{g}}_A, \quad \mathbb{E}[\mathbf{B}] = \mathbb{E}_j\left[\textstyle\sum_{i\neq j}\mathbf{g}_{B,i}\right] = \tfrac{1}{N}\textstyle\sum_{j=1}^N\sum_{i\neq j}\mathbf{g}_{B,i} = \tfrac{1}{N}\textstyle\sum_{i=1}^N(N-1)\mathbf{g}_{B,i} = (N-1)\bar{\mathbf{g}}_B, \tag{34}$$

$$\begin{aligned}\mathbb{E}[\mathbf{A}^T\mathbf{B}] &= \mathbb{E}_j\left[\mathbf{g}_{A,j}^T\left(\sum_{i\neq j}\mathbf{g}_{B,i}\right)\right] = \mathbb{E}_j\left[\mathbf{g}_{A,j}^T\left(\sum_{i=1}^N\mathbf{g}_{B,i} - \mathbf{g}_{B,j}\right)\right] = \mathbb{E}_j\left[\mathbf{g}_{A,j}^T(N\bar{\mathbf{g}}_B) - \mathbf{g}_{A,j}^T\mathbf{g}_{B,j}\right] \\ &= \mathbb{E}_j[\mathbf{g}_{A,j}]^T(N\bar{\mathbf{g}}_B) - \mathbb{E}_j[\mathbf{g}_{A,j}^T\mathbf{g}_{B,j}] = N\bar{\mathbf{g}}_A^T\bar{\mathbf{g}}_B - \mathbb{E}_j[\mathbf{g}_{A,j}^T\mathbf{g}_{B,j}],\end{aligned} \tag{35}$$

$$\begin{aligned}\text{Cov}(\mathbf{A},\mathbf{B}) &= \left(N\bar{\mathbf{g}}_A^T\bar{\mathbf{g}}_B - \mathbb{E}_j[\mathbf{g}_{A,j}^T\mathbf{g}_{B,j}]\right) - (\bar{\mathbf{g}}_A^T)((N-1)\bar{\mathbf{g}}_B) = \bar{\mathbf{g}}_A^T\bar{\mathbf{g}}_B - \mathbb{E}_j[\mathbf{g}_{A,j}^T\mathbf{g}_{B,j}] \\ &= -\left(\mathbb{E}_j[\mathbf{g}_{A,j}^T\mathbf{g}_{B,j}] - \bar{\mathbf{g}}_A^T\bar{\mathbf{g}}_B\right) = -\text{Cov}_j(\mathbf{g}_{A,j}, \mathbf{g}_{B,j}),\end{aligned} \tag{36}$$

Since $\mathbf{g}_{A,i}$ and $\mathbf{g}_{B,i}$ are positively correlated, i.e., $\text{Cov}_j(\mathbf{g}_{A,j}, \mathbf{g}_{B,j}) > 0$, i.e., $\text{Cov}(\mathbf{A},\mathbf{B}) < 0$. This proves that the CARL indeed reduces the gradient variance introduced by the component sampling of c_j.

E Data Availability

To facilitate reproducibility, we will release an anonymous artifact containing a reference implementation of Bifrost, the scripts to preprocess data, and the experimental pipeline used to generate the results reported in this paper. The artifact is available at https://anonymous.4open.science/r/Bifrost. All public datasets used in our evaluation are obtained from LogHub [60] and can be accessed following the instructions provided by the dataset maintainers. The only exception is Platform-X, which consists of Company-X's production business logs. Due to Company-X's privacy and confidentiality policies, Platform-X cannot be disclosed during the review period. We therefore treat the Platform-X results as an industrial case study and explicitly separate claims that rely on Platform-X from those supported by publicly available datasets.

References

[1] Anunay Amar and Peter C Rigby. 2019. Mining historical test logs to predict bugs and localize faults in the test logs. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 140–151.

[2] Ching-Yao Chuang, Joshua Robinson, Yen-Chen Lin, Antonio Torralba, and Stefanie Jegelka. 2020. Debiased contrastive learning. *Advances in neural information processing systems* 33 (2020), 8765–8775.

[3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 NAACL*. 4171–4186.

[4] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 1285–1298.

[5] Chiming Duan, Yong Yang, Tong Jia, Guiyang Liu, Jinbu Liu, Huxing Zhang, Qi Zhou, Ying Li, and Gang Huang. 2025. FAMOS: Fault diagnosis for Microservice Systems through Effective Multi-modal Data Fusion. In *ICSE 2025*. IEEE Computer Society, 610–610.

[6] Hongcheng Guo, Jian Yang, Jiaheng Liu, Jiaqi Bai, Boyang Wang, Zhoujun Li, Tieqiao Zheng, Bo Zhang, Junran Peng, and Qi Tian. 2024. Logformer: A pre-train and tuning pipeline for log anomaly detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 135–143.

[7] Minghua He, Chiming Duan, Pei Xiao, Tong Jia, Siyu Yu, Lingzhe Zhang, Weijie Hong, Jin Han, Yifan Wu, Ying Li, et al. 2025. United We Stand: Towards End-to-End Log-based Fault Diagnosis via Interactive Multi-Task Learning. *arXiv preprint arXiv:2509.24364* (2025).

[8] Minghua He, Tong Jia, Chiming Duan, Huaqian Cai, Ying Li, and Gang Huang. 2024. LLMeLog: An Approach for Anomaly Detection based on LLM-enriched Log Events. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 132–143.

[9] Minghua He, Tong Jia, Chiming Duan, Huaqian Cai, Ying Li, and Gang Huang. 2025. Weakly-supervised Log-based Anomaly Detection with Inexact Labels via Multi-instance Learning. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 726–726.

[10] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R Lyu. 2017. Drain: An online log parsing approach with fixed depth tree. In *2017 ICWS*. IEEE, 33–40.

[11] Junjie Huang, Minghua He, Jinyang Liu, Yintong Huo, Domenico Bianculli, and Michael R Lyu. 2025. CodeAD: Synthesize Code of Rules for Log-based Anomaly Detection with LLMs. *arXiv preprint arXiv:2510.22986* (2025).

[12] Junjie Huang, Zhihan Jiang, Zhuangbin Chen, and Michael Lyu. 2025. No More Labelled Examples? An Unsupervised Log Parser with LLMs. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2406–2429.

[13] Tong Jia, Ying Li, Yong Yang, and Gang Huang. 2024. Hilogx: noise-aware log-based anomaly detection with human feedback. *The VLDB Journal* 33, 3 (2024), 883–900.

[14] Miao Jiang, Mohammad A Munawar, Thomas Reidemeister, and Paul AS Ward. 2011. Efficient fault detection and diagnosis in complex software systems with information-theoretic monitoring. *IEEE Transactions on Dependable and Secure Computing* 8, 4 (2011), 510–522.

[15] Zhihan Jiang, Junjie Huang, Guangba Yu, Zhuangbin Chen, Yichen Li, Renyi Zhong, Cong Feng, Yongqiang Yang, Zengyin Yang, and Michael Lyu. 2025. L4: Diagnosing large-scale llm training failures via automated log analysis. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 51–63.

[16] Mengyuan Jing, Yanmin Zhu, Tianzi Zang, and Ke Wang. 2023. Contrastive self-supervised learning in recommender systems: A survey. *ACM Transactions on Information Systems* 42, 2 (2023), 1–39.

[17] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. 2020. Supervised contrastive learning. *Advances in neural information processing systems* 33 (2020),

18661–18673.
[18] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. [n. d.]. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. In *International Conference on Learning Representations*.
[19] Van-Hoang Le and Hongyu Zhang. 2021. Log-based anomaly detection without log parsing. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 492–504.
[20] Van-Hoang Le and Hongyu Zhang. 2022. Log-based anomaly detection with deep learning: How far are we?. In *Proceedings of the 44th international conference on software engineering*. 1356–1367.
[21] Van-Hoang Le and Hongyu Zhang. 2024. Prelog: A pre-trained model for log analytics. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–28.
[22] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R Lyu. 2023. Eadro: An end-to-end troubleshooting framework for microservices on multi-source data. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1750–1762.
[23] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 7871–7880.
[24] Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. 2023. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281* (2023).
[25] Qingwei Lin, Tianci Li, Pu Zhao, Yudong Liu, Minghua Ma, Lingling Zheng, Murali Chintalapati, Bo Liu, Paul Wang, Hongyu Zhang, et al. 2023. Edits: An easy-to-difficult training strategy for cloud failure prediction. In *Companion Proceedings of the ACM Web Conference 2023*. 371–375.
[26] Xiao Liu, Fanjin Zhang, Zhenyu Hou, Li Mian, Zhaoyu Wang, Jing Zhang, and Jie Tang. 2021. Self-supervised learning: Generative or contrastive. *IEEE transactions on knowledge and data engineering* 35, 1 (2021), 857–876.
[27] Yuchen Liu, Yixuan Meng, Kaiyuan Xu, Zijun Xu, Tianyuan Wu, Yiwei Yang, and Shu Yin. 2021. Reproducibility: Performance Evaluation of MemXCT on Azure CycleCloud Platform. *IEEE Transactions on Parallel and Distributed Systems* 33, 9 (2021), 2047–2049.
[28] Chuan Luo, Pu Zhao, Bo Qiao, Youjiang Wu, Hongyu Zhang, Wei Wu, Weihai Lu, Yingnong Dang, Saravanakumar Rajmohan, Qingwei Lin, et al. 2021. NTAM: neighborhood-temporal attention model for disk failure prediction in cloud platforms. In *Proceedings of the Web Conference 2021*. 1181–1191.
[29] Lipeng Ma, Weidong Yang, Bo Xu, Sihang Jiang, Ben Fei, Jiaqing Liang, Mingjie Zhou, and Yanghua Xiao. 2024. Knowlog: Knowledge enhanced pre-trained language model for log understanding. In *Proceedings of the 46th ieee/acm international conference on software engineering*. 1–13.
[30] Rui Meng, Ye Liu, Shafiq Rayhan Joty, Caiming Xiong, Yingbo Zhou, and Semih Yavuz. 2024. SFR-Embedding-Mistral:Enhance Text Retrieval with Transfer Learning. Salesforce AI Research Blog. https://www.salesforce.com/blog/sfr-embedding/
[31] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, et al. 2019. Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs.. In *IJCAI*, Vol. 19. 4739–4745.
[32] George A Miller. 1995. WordNet: a lexical database for English. *Commun. ACM* 38, 11 (1995), 39–41.
[33] Shuyi Niu, Jiawei Jin, Xiutian Huang, Yonggeng Wang, Wenhao Xu, and Youyong Kong. 2023. Locating Faulty Applications via Semantic and Topology Estimation. In *Companion Proceedings of the ACM Web Conference 2023*. 528–532.
[34] Adam Oliner and Jon Stearley. 2007. What supercomputers say: A study of five system logs. In *37th annual IEEE/IFIP international conference on dependable systems and networks (DSN'07)*. IEEE, 575–584.
[35] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).
[36] Jeffrey Pennington, Richard Socher, and Christopher D Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 1532–1543.
[37] Thomas Reidemeister, Mohammad A Munawar, and Paul AS Ward. 2010. Identifying symptoms of recurrent faults in log files of distributed information systems. In *2010 IEEE Network Operations and Management Symposium-NOMS 2010*. IEEE, 187–194.
[38] Carl Martin Rosenberg and Leon Moonen. 2020. Spectrum-based log diagnosis. In *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 1–12.
[39] Yicheng Sui, Yuzhe Zhang, Jianjun Sun, Ting Xu, Shenglin Zhang, Zhengdan Li, Yongqian Sun, Fangrui Guo, Junyu Shen, Yuzhi Zhang, et al. 2023. Logkg: Log failure diagnosis through knowledge graph. *IEEE Transactions on Services Computing* 16, 5 (2023), 3493–3507.

[40] Jinrui Sun, Tong Jia, Minghua He, Yihan Wu, Ying Li, and Gang Huang. 2025. Exploring Variable Potential for LLM-based Log Parsing Efficiency and Reduced Costs. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 596–600.

[41] Jia Tong, Li Ying, Tang Hongyan, and Wu Zhonghai. 2016. An approach to pinpointing bug-induced failure in logs of open cloud platforms. In *2016 IEEE 9th International Conference on Cloud Computing (CLOUD)*. IEEE, 294–302.

[42] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. 2023. Improving Text Embeddings with Large Language Models. *arXiv preprint arXiv:2401.00368* (2023).

[43] Lingzhi Wang, Nengwen Zhao, Junjie Chen, Pinnong Li, Wenchi Zhang, and Kaixin Sui. 2020. Root-cause metric location for microservice systems via log anomaly detection. In *2020 IEEE international conference on web services (ICWS)*. IEEE, 142–150.

[44] Thorsten Wittkopp, Philipp Wiesner, and Odej Kao. 2024. LogRCA: Log-Based Root Cause Analysis for Distributed Services. In *European Conference on Parallel Processing*. Springer, 362–376.

[45] W Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A survey on software fault localization. *IEEE Transactions on Software Engineering* 42, 8 (2016), 707–740.

[46] Tianyuan Wu, Wei Wang, Yinghao Yu, Siran Yang, Wenchao Wu, Qinkai Duan, Guodong Yang, Jiamang Wang, Lin Qu, and Liping Zhang. 2024. FALCON: Pinpointing and mitigating stragglers for large-scale hybrid-parallel training. *arXiv preprint arXiv:2410.12588* (2024).

[47] Xingfang Wu, Heng Li, and Foutse Khomh. 2023. On the effectiveness of log representation for log-based anomaly detection. *Empirical Software Engineering* 28, 6 (2023), 137.

[48] Wensheng Xia, Ying Li, Tong Jia, and Zhonghai Wu. 2019. Bugidentifier: An approach to identifying bugs via log mining for accelerating bug reporting stage. In *2019 IEEE 19th International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 167–175.

[49] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[50] Lin Yang, Junjie Chen, Zan Wang, Weijing Wang, Jiajun Jiang, Xuyuan Dong, and Wenbin Zhang. 2021. Semi-supervised log-based anomaly detection via probabilistic label estimation. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1448–1460.

[51] Qingyang Yu, Changhua Pei, Bowen Hao, Mingjie Li, Zeyan Li, Shenglin Zhang, Xianglin Lu, Rui Wang, Jiaqi Li, Zhenyu Wu, et al. 2023. Cmdiagnostor: An ambiguity-aware root cause localization approach based on call metric data. In *Proceedings of the ACM web conference 2023*. 2937–2947.

[52] Chenyangguang Zhang, Tong Jia, Guopeng Shen, Pinyan Zhu, and Ying Li. 2024. MetaLog: Generalizable Cross-System Anomaly Detection from Logs with Meta-Learning. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.

[53] Dun Zhang, Jiacheng Li, Ziyang Zeng, and Fulong Wang. 2025. Jasper and Stella: distillation of SOTA embedding models. arXiv:2412.19048 [cs.IR] https://arxiv.org/abs/2412.19048

[54] Lingzhe Zhang, Liancheng Fang, Chiming Duan, Minghua He, Leyi Pan, Pei Xiao, Shiyu Huang, Yunpeng Zhai, Xuming Hu, Philip S Yu, et al. 2025. A Survey on Parallel Text Generation: From Parallel Decoding to Diffusion Language Models. *arXiv preprint arXiv:2508.08712* (2025).

[55] Lingzhe Zhang, Tong Jia, Kangjin Wang, Weijie Hong, Chiming Duan, Minghua He, and Ying Li. 2025. Adaptive root cause localization for microservice systems with multi-agent recursion-of-thought. *arXiv preprint arXiv:2508.20370* (2025).

[56] Shenglin Zhang, Sibo Xia, Wenzhao Fan, Binpeng Shi, Xiao Xiong, Zhenyu Zhong, Minghua Ma, Yongqian Sun, and Dan Pei. 2024. Failure Diagnosis in Microservice Systems: A Comprehensive Survey and Analysis. *arXiv preprint arXiv:2407.01710* (2024).

[57] Xu Zhang, Yong Xu, Qingwei Lin, Bo Qiao, Hongyu Zhang, Yingnong Dang, Chunyu Xie, Xinsheng Yang, Qian Cheng, Ze Li, et al. 2019. Robust log-based anomaly detection on unstable log data. In *Proceedings of the 2019 27th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*. 807–817.

[58] Xu Zhang, Yong Xu, Si Qin, Shilin He, Bo Qiao, Ze Li, Hongyu Zhang, Xukun Li, Yingnong Dang, Qingwei Lin, et al. 2021. Onion: identifying incident-indicating logs for cloud systems. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1253–1263.

[59] Yingying Zhang, Zhengxiong Guan, Huajie Qian, Leili Xu, Hengbo Liu, Qingsong Wen, Liang Sun, Junwei Jiang, Lunting Fan, and Min Ke. 2021. CloudRCA: A root cause analysis framework for cloud computing platforms. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 4373–4382.

[60] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R Lyu. 2023. Loghub: A large collection of system log datasets for ai-driven log analytics. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 355–366.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009